Contain, Conform, Compare (Extended Version): A Red-Teamed Evaluation Kit for Small Coding Agents and the Full Record of Its Experiments

Nguyen Minh Thang
VNU University of Science, 334 Nguyen Trai, Thanh Xuan, Hanoi, Vietnam

November 2026

Abstract

A hosted score in the Gemma 4 Developer Agent competition [16] is one noisy number, so development needs a local evaluator that is fast, safe for agent-written code, and matching the hosted scorer's prompts. `drydock-eval` evaluates a small coding agent (Gemma 4 E4B [12]) at about 250 runs per hour on one RTX 5090. *Contain* runs each job in a fresh container without network and passed 200 scripted escape tests; *conform* provides eight checks that compare a local stack's requests with the scorer's configuration, down to rendered tokens; *compare* provides paired statistics that respect repository clusters. AI agents built and attacked the kit; a human owner approved machine use and release. This extended version repeats the short paper and adds the full record of the experiments made with and around the kit: 9,660 rows of agent runs, 2,494 grading controls and 72 serving measurements, all released. On four L4 GPUs the 31B model decodes 38.1 tokens per second in one stream and E4B 105.8. Six configurations of a 31B agent solved between 37 and 44 of 70 development tasks, and no pairwise difference was detected. A pre-registered five-arm experiment (231 tasks, 4,620 jobs) measured four prompt divergences: no E4B solve-rate change was detected (largest estimate -2.1 points, 95% interval -4.4 to $+0.4$). Three changes of the E4B agent led their screens by $+1.0$ to $+6.2$ points; each was then tested once on new tasks under a frozen pass rule, and none was confirmed (gains -0.35, -0.39 and +4.10 points). The unchanged baseline, run again on the same 128 tasks, moved by 1.17 points, and 65% of these tasks were solved in none of eight runs. Prompts and agent bundles are withheld; every run-level result is public.

1 Introduction

The competition scores an agent bundle on hidden GitHub-issue tasks, with Gemma 4 served by vLLM [23]. Third-party analyses put the noise of one hosted score near 1.5 of 58 tasks [7, 29]. With one submission per day, a hosted score cannot separate two agents that differ by a few tasks. Local evaluation must therefore carry development; on a Kaggle kernel with four L4 GPUs our stack finished 35.2 agent runs per hour, serially. A local evaluator has to answer three questions before its numbers mean anything.

Safety. An agent run executes model-written shell commands and installs model-edited packages; our first local setup ran them with no sandbox.

Fidelity. We call a *rendering trap* a configuration detail that silently changes the token sequence the model receives while runs still finish and score. The *scorer path* is the scorer's configuration reproduced locally. In our first setup the served model name contained "gemma4", so the agent framework (google-adk 1.36.1) sent tool results with role `tool_responses` instead of `tool` [35]. Prompts rebuilt from its traces matched the server's token counts on 106 of 114 calls only when rendered with `tool_responses` (0 of 114 with `tool`). Solve counts hid it: that setup solved 3 and 5 of 24 tasks against 4 and 4 on Kaggle (Fisher p 0.74 and 0.75).

These traps are public: the role rule [35], thought handling in history [9, 10, 15], a newline after thinking text [13], and the scorer's compaction settings [36]. We discovered none of them; we contribute checks that detect them (Section 5) and a measurement of three of them plus a template swap (Section 9).

Inference. Tasks from one repository succeed and fail together, a small model solves about one task in five, and a development team looks at many variants. A difference between two arms then needs a paired design, an analysis that respects repositories, and a rule that is fixed before the outcomes are read.

What this version adds. This is the extended version of a paper of 3,000 words written for the competition's paper track [27]. Sections 4 to 6 and 9.1 repeat the text of that paper, and Sections 9 and 10 include the text of its supplement. Everything else is new here. The short paper describes the kit and one experiment. It leaves out most of what was measured while the kit was built and used: the throughput of the serving stack, the experiments with the 31B model that came before the work on the small model, the first experiments with E4B, the runs that compared the workstation with the hosted notebook, and the three candidate changes of the E4B agent that were taken through a screen and a confirmation. This version reports all of them, with every run as a released row.

Contributions.

- **A kit.** `drydock-eval` has three modules. *Contain* is a container runner for agent jobs together with the record of the attacks that seven versions of it failed (Section 4). *Conform* is a set of eight checks that compare the requests of a local stack with the scorer path (Section 5). *Compare* is a set of paired tests with repositories as clusters and a pass rule whose behaviour was simulated before use (Section 6).
- **A measurement of four prompt divergences** on E4B in a pre-registered experiment with five arms, 231 tasks and 4,620 jobs (Section 9).
- **Serving measurements** of three Gemma 4 models on the hardware type of the scorer, including the cost of serving a LoRA adapter (Section 3).
- **The record of 5,024 further agent runs** in 31 blocks and 37 arms: 710 with the 31B model (Section 7) and 4,314 with E4B (Sections 8 and 10), together with 2,494 control gradings of the task pool.
- **Three complete screen-and-confirm programmes** for a change of the E4B agent, each with its selection step, its frozen rule and its outcome (Section 10), and an analysis of what they say about noise and selection (Section 11).
- **Data and scripts.** Every table and figure of Sections 3, 7, 8, 10 and 11 is computed by a released script from released files (Section 14).

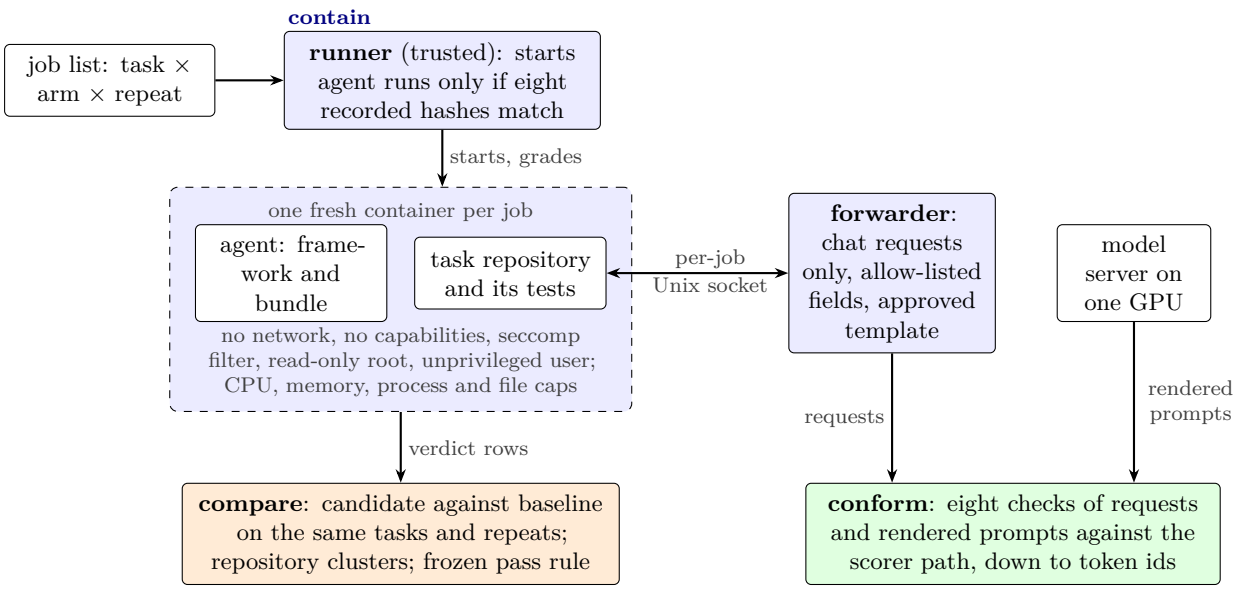


Figure 1: The kit: where each module sits.

Main findings. Most of the results are null results, and we report them as such.

- No attack on the runner escaped the container's namespaces or privileges, but nine demonstrated high-severity vectors came through what the host shared with it, in designs that passed their own escape tests.
- Four prompt divergences from the scorer path changed the E4B solve rate by less than the experiment could detect; its simultaneous intervals leave room for a drop of up to 5.1 points and a gain of up to 3.5 points.
- Six configurations of a 31B agent, from a public single agent to a wrapped agent with a shell-only coder, solved 37 to 44 of the same 70 tasks; no pairwise difference was detected, and 25 of the 70 tasks were solved by some configurations and not by others.
- Each of three E4B candidates led its screen. On new tasks the first two had estimated gains below zero. The third gained +4.10 points, with a one-sided p of 0.0405 against a required 0.025: not confirmed.
- When the unchanged baseline ran again on the same 128 tasks, its total moved by 1.17 points. 83 of these tasks were solved in none of eight runs and 10 in all eight, so the remaining 35 tasks carry all of the repeat-to-repeat noise.

Section 2 gives the setting and the terms. Section 3 reports the serving measurements. Sections 4 to 6 describe the three modules. Sections 7 and 8 report the experiments with the two models, Section 9 the case study, and Section 10 the three candidate programmes. Section 11 draws the experiments together; related work, limitations and availability follow.

2 Setting, terms and rules

2.1 The competition and its scorer

A submission to the competition is an *agent bundle*: a directory of configuration files for the google-adk agent framework, which names the agents, their prompts, their tools, their sampling settings

and their limits. The base model is not part of the bundle: the scorer loads the bundle into the competition's harness, serves one Gemma 4 model with vLLM on four L4 GPUs, and runs the agent on hidden tasks one after another. A task is an issue of a public Python repository at a fixed commit. The agent works in a sandbox with the repository checked out, and ends by submitting a patch. The task counts as solved if hidden tests pass after the patch is applied. The whole scoring run has a time limit of 12 hours, so a slow agent loses tasks at the end. A team may submit once per day, and the public leaderboard shows the solved share on 58 of the hidden tasks.

Two properties of this setting drive the rest of the paper. First, one hosted score is a draw with a standard deviation near 1.5 tasks of 58 [7, 29], and there is one draw per day. Second, the scorer's configuration is public in a starter notebook [36], but a local stack can differ from it in ways that do not stop a run (Section 5).

2.2 Models and software

Stack. Gemma 4 E4B, 4-bit quantization-aware checkpoint (Kaggle model version 2); vLLM 0.19.1; google-adk 1.36.1; the competition's harness packages. The experiments of Section 7 use the 31B model, also as a 4-bit quantization-aware checkpoint. The serving measurements of Section 3 add the 12B model. We write 31B, 12B and E4B for the three models. E4B is one of the two models that are eligible for the small-model prize of the competition, and it is the model of the kit's main use.

2.3 Machines

Agent runs were made on three kinds of machine, and every released row names its kind.

Emulated latency. The first experiments with the 31B model ran on a rented cloud machine with one large GPU. Its server answers faster than the scorer's four L4 GPUs, so each model call was padded to the duration that a latency model predicts for the scorer's hardware: $0.15 + 0.032\,c + 0.0007\,n$ seconds for a call that generates c tokens after n new prompt tokens. We fitted this model to public call logs from real four-L4 hardware [30]. Several jobs ran at the same time, and the load differed between blocks; this is why arms are compared within a block wherever possible. Section 7.5 compares the emulation with real hardware.

Four L4 GPUs. Kaggle notebooks with four L4 GPUs are the hardware type of the scorer. Jobs ran there with the server flags of the starter notebook and without padding. Notebook hours are limited by a weekly quota, so these blocks are small.

Workstation. Experiments with E4B from Section 8.3 on ran on a shared single-GPU workstation (RTX 5090, 32 GB), with eight jobs at a time and each job in a container of the runner of Section 4. This is the machine for which the kit was built; it finishes about 250 runs per hour.

2.4 Tasks

Development tasks. The competition distributes development tasks with its harness. We used 70 of them, from four repositories: a first set of 30 and 40 further tasks. Competition data may not be redistributed, so the released rows name these tasks by opaque codes (`dev-001` and so on) and give only their repository.

Berth. Tasks for the work on E4B come from Berth [33]: 579 of its 653 tasks load in the harness and pass our validation (the reference patch passes, an empty patch fails). The released rows keep the Berth task identifiers. Four task sets were drawn from these 579 tasks: a screen set S of 48 tasks, a small panel C of 72 tasks in 18 repositories, a large panel of 200 tasks in 20 repositories that contains C, and the case-study panel of 231 tasks in 15 repositories. We write H for the 128 tasks of the large panel that are outside C. S, the large panel and the case-study panel share no task. The two panels were frozen before any outcome on them existed (Section 6). S contains the 24 tasks of the parity runs of Section 8.3, on which results of the baseline were known before the screens ran; Section 10 comes back to this.

2.5 Terms

A *run* is one agent job on one task: the harness prepares the sandbox, the agent works until it submits or is stopped, and the harness grades the patch. A run is *solved* or not; a run that ends without a patch or by a limit counts as not solved. An *arm* is one agent configuration, that is, one bundle with its settings. A *block* is one session on one machine in which one or more arms ran over one task list in a single job queue; a block with several arms interleaves them, so they share the load and the time of day. A *repeat* is one of several runs of the same arm on the same task within a block. *Points* are percentage points of the solve rate. A *task-equivalent* is one task solved in every repeat, or the same number of solved runs spread over tasks. Arms carry neutral labels (Table 32); block labels name the experiment (Table 33).

Counts of solved runs come with 95% Wilson intervals where a single arm is described. Two arms on the same tasks with one run per task are compared by the exact two-sided sign test on the tasks that only one of them solved. With repeats, and for every decision about a candidate, the tests of Section 6 are used. A null result is worded "no difference was detected"; none of the experiments below was sized to show that two arms are equal.

2.6 Threat model

The adversary is the code the agent writes. It may try to read host files or other jobs, reach the network, exhaust host resources, forge its verdict, or outlive its job. Kernel exploits are out of scope.

2.7 Who did the work, and what is withheld

Who did the work. AI agents wrote the kit's code, attacks and analyses (builders). Separate AI agents (verifiers) re-implemented or re-ran the work adversarially. A lead AI session froze the protocols; a human owner approved isolation, machine time and release. "Verified" below means this process, not human review. The same holds for the experiments that this version adds: AI agents built the arms, ran the blocks and wrote the analysis scripts and this text, and the owner approved each hosted submission and each release.

What is withheld. The competition is still running. The E4B agent bundle that serves as the baseline of Sections 8 to 10, the changed bundles derived from it, their prompts, the fine-tuned adapter of Section 8.1 and its training data are withheld for now; the release of the bundle and of the request dumps of the case study is planned for 2026-12-03. The arms are therefore described by the kind of change and not by their text (Table 32). What is released is the outcome and the measurements of every run. Two 31B arms are public bundles run unchanged: `BF5` is the

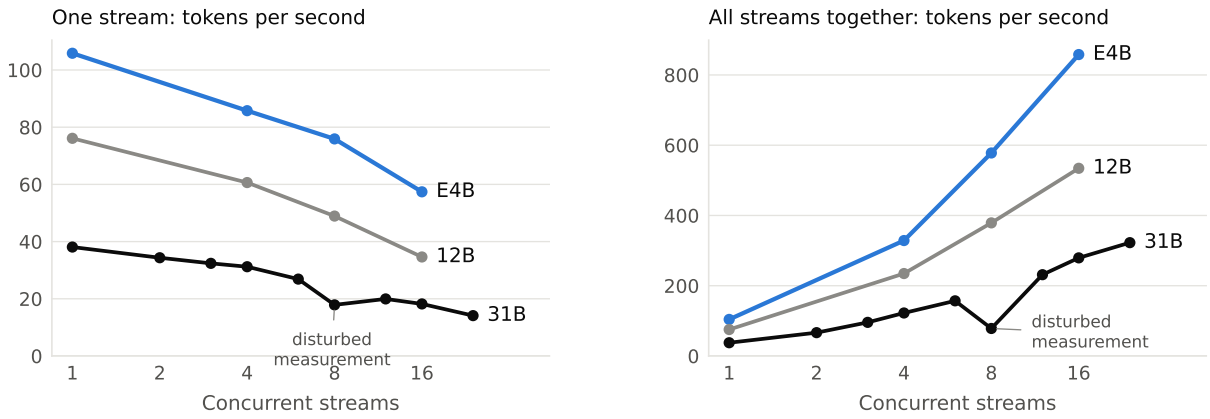


Figure 2: Decode throughput on four L4 GPUs with a context of 2,048 tokens. Left: tokens per second of one stream as the number of concurrent streams grows. Right: tokens per second summed over the streams.

public Budget-Fit V5 configuration, the most-run public configuration in the hosted-score census of dariushafshar (Kaggle user) [7], and `ID24` is a bundle of the public collection of Team Muhammed Idris [39].

3 Serving measurements

How fast the scorer's server generates tokens decides how many model calls an agent can afford per task, and whether more than one agent can work on a task at the same time. We measured it before designing any agent.

Method. A Kaggle notebook with four L4 GPUs started the vLLM server with the flags of the starter notebook [36]: the model split over the four GPUs, a maximum context of 32,768 tokens, and the Gemma 4 parsers. A client then sent streaming completion requests with prompts of random tokens. For each prompt length and each number of concurrent streams, a first request generated one token, which measures the processing of the prompt (prefill) and fills the prefix cache; a second request with the same prompt generated 160 tokens, which measures decoding on a cached prefix. The 31B, 12B and E4B models were measured one after another, each with its own server. A second session repeated the 31B measurement with a LoRA adapter loaded. The 72 measured cells and the server records are in `serving_bench.csv` and `serving_servers.csv`.

One stream. The 31B model decodes 38.1 tokens per second with a short context and 35.5 with 14,336 tokens of context (Table 1). The 12B model reaches 76.1 and E4B 105.8 tokens per second, 2.8 times the 31B model. Prefill is fast in comparison: about 1,539 tokens per second for 31B. An agent that keeps its prompt prefix stable pays this once, because the server reuses cached prefixes. Generated tokens are what costs time: a 31B call that writes 1,000 tokens takes about half a minute. The server needs 337 s to start with the 31B model, which counts against the time limit of a scoring run.

Several streams. Concurrent streams share the GPUs (Figure 2). With four streams and a long context, each 31B stream still decodes 27.9 tokens per second, 107 in total. Apart from one cell

Table 1: Serving throughput on four L4 GPUs (vLLM, one server per model). Context: prompt tokens already in the request. Streams: concurrent requests. Each request generates 160 tokens.

Model	Context	Streams	Prefill tok/s	Decode tok/s per stream	Decode tok/s, all streams
31B	2,048	1	1,539	38.1	38
31B	2,048	2	1,545	34.4	66
31B	2,048	4	1,546	31.2	122
31B	2,048	8	1,550	17.9	78
31B	2,048	16	1,548	18.2	279
31B	8,192	1	1,479	36.3	36
31B	8,192	2	1,480	33.7	66
31B	8,192	4	1,485	29.3	114
31B	8,192	8	1,486	9.2	25
31B	14,336	1	1,426	35.5	35
31B	14,336	2	1,427	32.8	64
31B	14,336	4	1,429	27.9	107
12B	2,048	1	3,054	76.1	75
12B	2,048	4	3,076	60.6	235
12B	2,048	8	3,089	48.9	379
12B	2,048	16	3,082	34.6	534
12B	8,192	1	2,973	69.8	69
12B	8,192	2	2,994	63.2	123
12B	8,192	4	3,002	55.0	213
E4B	2,048	1	5,267	105.8	104
E4B	2,048	4	5,385	85.8	329
E4B	2,048	8	5,372	75.9	578
E4B	2,048	16	5,404	57.4	858
E4B	8,192	1	5,212	95.9	94
E4B	8,192	2	5,235	85.0	164
E4B	8,192	4	5,254	73.5	282

discussed below, the total grows up to 16 streams for all three models, to 279 tokens per second for 31B and 858 for E4B at 2,048 tokens of context. The scorer runs tasks one after another, so this capacity is only useful to an agent that runs several model calls in parallel within one task.

Two cells that do not fit. With eight streams of 8,192 tokens, the 31B server delivered 9.2 tokens per second per stream, a third of the four-stream value. The server reported a key-value cache of 46,352 tokens for 31B, against 160,416 for 12B and 298,464 for E4B, and eight contexts of 8,192 tokens hold 65,536 tokens. We read the drop as cached prefixes being evicted and processed again; we did not test this reading. With eight streams of 2,048 tokens the 31B server gave 17.9 tokens per second per stream and 78 in total, less in total than with four streams (122) and with 16 (279). We have no explanation for this cell and report it as measured; Figure 2 marks it.

Cost of an adapter. The competition allows LoRA adapters on the one served model. With an adapter of rank 16 or rank 8 loaded, requests addressed to the adapter decoded 9 to 13% slower than requests to the same server without an adapter (Table 2), and the reported key-value cache shrank by 19% to 37,328 tokens. Requests addressed to the base model on a server with a loaded adapter changed by -1.1 to -0.9% (one stream, short context). Which layers the adapter covers made no visible difference. The measurements were made once per cell.

Table 2: Cost of serving a LoRA adapter with the 31B model (same server settings; requests addressed to the adapter). Decode speed in tokens per second per stream; change against the control in brackets.

Server	KV cache, tokens	2k context, 1 stream	8k, 1 stream	8k, 4 streams
no adapter (control)	46,352	38.3	36.4	29.4
rank 16, attention projections	37,328	33.9 (−12%)	32.4 (−11%)	26.9 (−9%)
rank 16, all linear layers	37,328	33.8 (−12%)	32.5 (−11%)	26.9 (−9%)
rank 8, two module types	37,568	33.3 (−13%)	32.1 (−12%)	26.7 (−9%)

Table 3: Runner versions.

Runner version	Canaries passed	Security findings of independent verifiers
1. Container per command	52/52	4 high: host-side install ran the agent's `setup.py` (builder-found); symlink leaked a host file; 10 GiB of host disk reserved in 0.005 s; host copied a sparse file in full
2. Hardened, per command	60/60 (57/60 before canary fixes)	4 high, 1 untested: host overwrote a file through a planted link; cleanup followed links; 6 GiB reserved in 0.012 s; 200,000 files in 2.4 s
3. Container per job	98/98	1 high: the model server could fetch URLs named in requests; 2 medium
4–6. Request filter added, tightened	134/134, 159/159, 191/191	medium at most
7. Final	200/200	no high or medium

What followed for the design. At 35 tokens per second, an agent with a limit of eight minutes per task, the limit of the public configuration `BF5`, can generate at most about 17,000 tokens, before the time of its tool calls and of prefill is taken off. A second agent that works in parallel slows the first by about a tenth at two streams. E4B generates 2.8 times as fast, so the same task limit is rarely what stops an E4B agent (Section 8.6).

4 Contain: a runner and the attacks it failed

In the final design each job gets one fresh container: no network, no capabilities, a seccomp filter, a read-only root, an unprivileged user, 2 CPUs, 10 GiB of memory, 4,096 processes, and temporary filesystems capped in bytes and inodes. The only request channel is a per-job Unix socket to a forwarder, which passes only chat-completion requests with allow-listed fields, text-only content and an approved template hash, under rate and size caps. Agent runs start only if eight recorded hashes (image, code, flags, model files) still match.

A *canary* is a scripted attack with a machine-checked outcome.

Passing canaries proved little. No attack escaped the container's namespaces or privileges in any round. Nine demonstrated high-severity vectors came through what the host shared with it: host-side steps that touched agent-written files, resource limits (a known class [11]) and the model server.

Final version. Controls on 481 tasks (reference patch 481/481, empty patch 0/481) showed no verdict, exit-code or test-summary difference from the previous version. After the runner was killed, the container was gone within 149 s with no further model request. The final runner completed a 24-job list at 251–254 runs per hour with 8 workers.

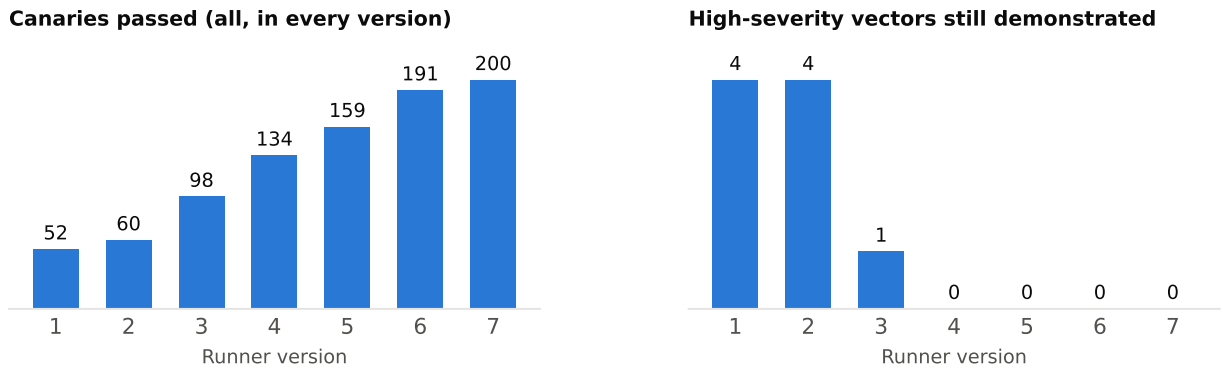


Figure 3: Canaries passed and high-severity vectors found, per runner version.

Table 4: The eight checks.

Check	Compares	Mutation runs caught
role	role of tool results	53/53
sampling	temperature, top-p, token and thinking budgets	52/52
tools	declared tool schemas	20/20
shape	message skeleton, system text, field encoding	72/72
template	rendered prompt against the Kaggle template	13/21 by token count, 24/24 by token id
thoughts	history thoughts, under the key the server reads	52/52
newline	newline before the token closing a thought	12/12
compaction	when history is summarized (14,336 tokens [36])	19/19

What remains. Containers share the host kernel; user namespaces are not remapped; the request parser runs in the trusted runner; a job can slow others through the shared model server within its caps. Our claim is conditional: against the stated adversary and with the eight hashes matching, no tested attack succeeded.

5 Conform: eight checks against the scorer path

`conform` reads the requests a local stack sent and, where logged, the prompts the server rendered. Reference values come from the public starter notebook [36], our Kaggle-kernel traces, and the Kaggle model's chat template.

Role. From each google-adk release's source (only 1.36.1 was executed): 1.34.0–1.39.1 and 2.0–2.4 send `tool_responses` when the model string contains "gemma4"; 2.5–2.11 when it matches `gemma-?4`; 2.12–2.14 exempt three OpenAI-compatible providers [35]. The harness packages require `>=1.34.0,<2.0`.

Thoughts. vLLM 0.19.1 reads an earlier assistant thought only from the field `reasoning`. google-adk 1.36.1 writes `reasoning_content`, which that server drops silently [9]; the competition harness writes both. Removing `reasoning` in a probe lost 55 of 55 thoughts from the rendered prompt.

Table 5: Simulated pass rates (3,000 simulations per cell; 400 for the dropping row).

Simulated scenario	Small panel	Large panel
No effect, exchangeable tasks	0.23–1.90%	0.00–0.50%
No mean effect, candidate breaks 20% of repositories	7.50%	8.43%
No effect, worst tasks dropped after seeing outcomes	31.75%	40.75%
Even gain of 3/58 (power)	≈60%	≈90%
Gain confined to four repositories (power)	15.9%	24.9%

Template. The check re-renders each request outside vLLM with the Kaggle template and compares token counts with the server's, or token ids when the server logs them, because identical strings can tokenize differently [40].

Evidence. In version 2's 24-task parity run, all 2,566 tool results had role `tool`, and re-rendering reproduced the server's token count on 396 of 396 requests (with `tool_responses`: 0 of 358 multi-turn requests). With the final runner, the checker reproduced the token ids of 243 of 243 returned calls, and a renderer written separately by the verifier matched 50 of 50 scorer-path requests. Prompts rebuilt from early calls of our Kaggle-kernel traces match the recorded counts on 181 of 189 calls with role `tool` and thoughts kept, 1 of 189 with `tool_responses` and 0 of 189 with thoughts dropped: our only Kaggle-side evidence, and count-level.

Mutation suite. 103 mutations (85 drifts, 18 benign controls) in four scripted scenarios against a mock server give 412 runs. The intended check caught 317 of 325 applicable drift runs; the 8 misses are same-length template changes seen through counts only. No check failed in 72 control runs (24 warned).

6 Compare: a decision rule that respects repositories

Tasks from one repository succeed and fail together. Clustered inference for model evaluations is prior art [28]; `compare` packages a small-cluster version with a frozen pass rule and its simulated behaviour.

Candidate and baseline run on the same tasks and repeats. A candidate passes only if (1) the mean gain is at least 2/58 (3.45 points, two leaderboard tasks); (2) one-sided $p \leq 0.025$ under a null-imposed wild cluster bootstrap-t over repositories, with Rademacher weights and 9,999 draws [4, 24]; (3) at least three repositories gain and every leave-one-repository-out estimate is positive; (4) agent time stays within 1.2 times baseline and infrastructure errors do not rise; and (5) no repository loses two task-equivalents or more. Two panels were frozen before any outcome: a small one (72 tasks, 18 repositories) and a large one (200 tasks, 20 repositories).

The 2.5% level holds for exchangeable effects, not for skewed ones, and not if tasks are dropped after unblinding. Independent re-implementations matched the rule code on 280 of 280 clean datasets (first version) and, with rule readings aligned, 193 of 194 (amended). Applied three times, to screen leaders from one separate 48-task set (25, 22, 25 of 96 runs solved; baselines 19, 21, 20); none passed. Gains: −0.35 points on the small panel ($p = 0.60$); on the 128 large-panel tasks outside it, −0.39 ($p = 0.60$), then +4.10 ($p = 0.04 > 0.025$, uncorrected). Condition (5): first application only.

The three applications are reported in full in Section 10, with the screens that selected their candidates.

7 Experiments with the 31B model

The work began with the 31B model, before the kit existed. The question was whether a different agent architecture solves more tasks than the public single-agent configuration within the scorer's time limit. The answer of 710 runs is that no difference was detected. These experiments also produced the habits that the kit later fixed in code: compare arms inside one block, keep task sets apart, and distrust a lead that was found by looking at many variants.

7.1 Arms

Table 32 lists every arm. The 31B arms fall into four families.

Reference. `BF5` is the public Budget-Fit V5 configuration, run unchanged: one agent with the tools of the harness, a limit of eight minutes and 28 tool calls per task, and the history compaction of the starter configuration, which replaces older turns by a summary when the history grows. `BF5-p`, `BF5-c` and `BF5-s` change one thing each: a section of the task prompt, a checking step before the patch is submitted, and the sampling settings.

Wrapped. In `W1` a root agent hands the task to a coder agent through the agent-as-tool mechanism of the framework. The coder runs with its own history, and the harness does not summarise it (column Sum. of Table 6). The first runs showed a defect: when the coder ended after a malformed tool call, the root submitted an empty patch. `W2` lets the root restart the coder a bounded number of times, and `W3` leaves the root with nothing else to do: it can pass the request on and restart the coder, and it cannot submit. `W1-b` and `W1-p` change limits and the coder prompt.

Shell only. In the `SH` arms the coder works through a shell tool alone and edits files with shell commands. `SH1`, `SH3` and `SH5` are successive versions of its prompt and limits; `SH6` adds an analysis step to the prompt of `SH5`.

A second public bundle. `ID24` is a public bundle [39] run unchanged, and `ID24-r` a variant with one tool removed.

7.2 The first 30 tasks

Block `pub31-1` ran three arms in one queue on 30 development tasks (Table 6). The wrapped arm `W1` solved 20 tasks and `BF5` 17: 3 tasks were solved by `W1` only and 0 by `BF5` only (sign test $p = 0.25$). `W1` was also faster, 254 against 303 seconds per job, because none of its runs spent time on history summaries, while 18 of the 30 `BF5` runs wrote at least one. A lead of three tasks on 30 is what one hopes to see and cannot trust.

Two attempts to improve `W1` by changing its limits and its prompt went the other way. `W1-b`, with a shorter time limit and more tool calls, solved 17, and `W1-p`, with a rewritten coder prompt and a shorter time limit, solved 15, with 11 runs stopped by the time limit and 12 without a patch. Both ran in blocks of their own, under a load that differed from that of the first block, so the comparison with `W1` mixes the change with the session. What can be said is that neither shorter limits nor the rewritten prompt gave more solved tasks.

Table 6 also shows `W3` and the first shell-only arm in one block. `SH1` solved 20 and `W3` 16 (4 tasks only `SH1`, 0 only `W3`; $p = 0.125$). `W3` has the coder of `W1`. In their two blocks `W3` solved 16 and

Table 6: The 31B arms on the development tasks, one run per task, with emulated L4 latency. Top: the first 30 tasks. Middle: 40 further tasks. Bottom: all 70. Wall: mean seconds per job. Out: mean generated tokens. T/O: runs stopped by the session time limit. Empty: runs that left no patch. Sum.: runs with at least one history summary.

Block	Arm	Tasks	Solved	%	95% interval	Wall	Out	Calls	T/O	Empty	Sum.
pub31-1	BF5	30	17	57	39–73	303	6,893	24.2	5	1	18
pub31-1	BF5-p	30	18	60	42–75	296	6,921	23.1	4	4	19
pub31-1	W1	30	20	67	49–81	254	5,942	22.4	3	4	0
pub31-2	W1-b	30	17	57	39–73	290	6,394	23.4	7	7	0
pub31-3	W1-p	30	15	50	33–67	304	6,412	25.2	11	12	0
pub31-4	W3	30	16	53	36–70	272	7,448	30.6	4	6	0
pub31-4	SH1	30	20	67	49–81	237	6,739	24.5	1	2	0
pub31-5	BF5	40	20	50	35–65	338	7,032	23.3	8	2	29
pub31-5	W2	40	19	48	33–63	312	6,582	24.4	7	6	0
pub31-6	W3	40	22	55	40–69	260	7,399	25.6	2	6	0
pub31-6	SH3	40	21	52	37–67	223	6,109	24.4	3	7	0
pub31-7	SH5	70	41	59	47–69	272	7,821	27.0	6	7	0
pub31-8	SH6	70	44	63	51–73	280	7,978	25.6	7	8	0

Table 7: Six configurations on all 70 development tasks (one run per task). The last three columns compare each configuration with BF5 on the same tasks: tasks only it solved, tasks only BF5 solved, and the two-sided exact sign test.

Configuration	First 30	Next 40	All 70	%	95% interval	Only it	Only BF5	p
BF5	17	20	37	53	41–64	—	—	—
W1/W2	20	19	39	56	44–67	6	4	0.75
W3	16	22	38	54	43–65	6	5	1.00
SH1/SH3	20	21	41	59	47–69	8	4	0.39
SH5	17	24	41	59	47–69	7	3	0.34
SH6	21	23	44	63	51–73	10	3	0.09

W1 20 of the same 30 tasks: 5 tasks only W1, 1 only W3 ($p = 0.22$). The data cannot say whether the changed root or the session accounts for this.

7.3 Forty further tasks and the total of seventy

The leads of the first 30 tasks were tested on 40 further tasks. BF5 solved 20 and the wrapped arm W2 19 (4 and 3 tasks solved by one arm only; $p = 1.00$). W3 solved 22 and the shell-only arm SH3 21 (2 and 1; $p = 1.00$). Neither lead of the first 30 tasks appeared again, although both leading arms had been revised in between.

Table 7 joins the blocks into six configurations that each ran once on all 70 tasks. Two of them join two versions of an arm: W1 on the first 30 tasks with W2 on the next 40, and SH1 with SH3. The six totals lie between 37 and 44 of 70 (Figure 4). Against BF5, the largest difference is that of SH6, with 10 tasks that only it solved and 3 that only BF5 solved ($p = 0.09$, not corrected for the five comparisons). No difference was detected. SH6 is also the last of a series of versions that were developed while looking at results on these same tasks, so its count is the one most exposed to selection.

Figure 5 shows where the configurations differ. 25 tasks were solved by every configuration and 20 by none; on the remaining 25 tasks the outcome depends on the configuration or on the run.

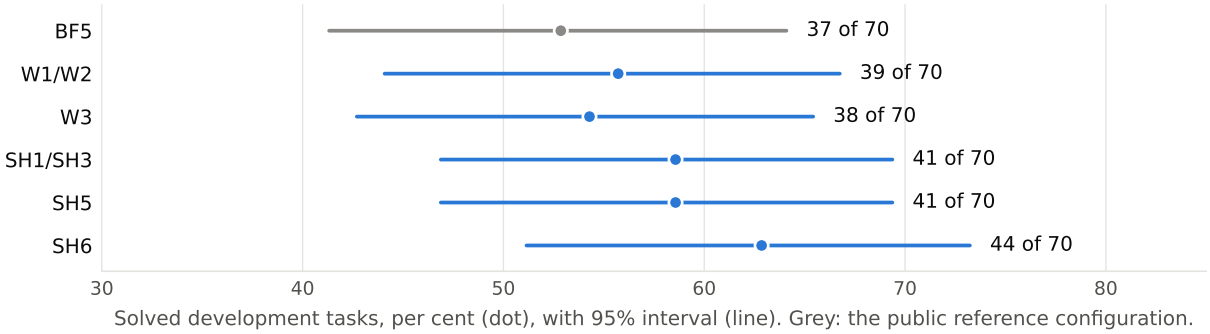


Figure 4: Six configurations of the 31B agent on the same 70 development tasks, one run per task.

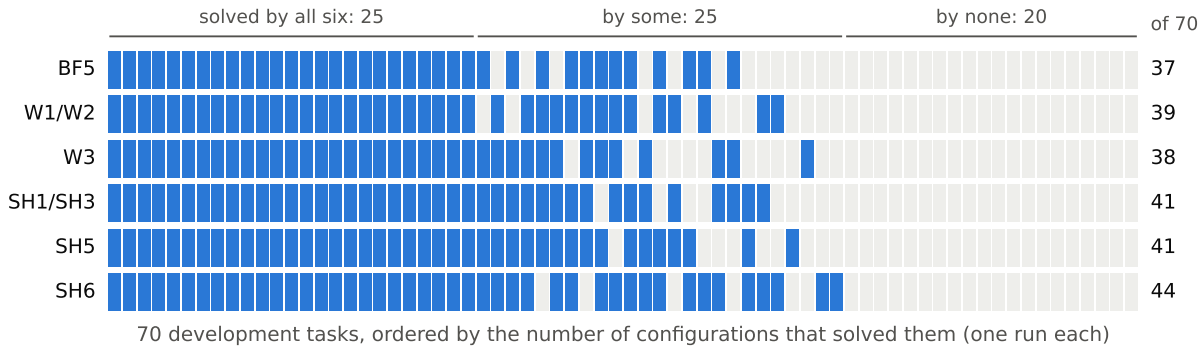


Figure 5: Which configuration solved which task. One column per development task, one row per configuration; a filled cell is a solved run. 25 tasks were solved by all six configurations and 20 by none.

The union of the six is 50 tasks, against 44 for the best single one. This gap is an upper bound on what a choice between configurations could gain, and it is inflated: a task that one configuration solves in half of its runs enters the union easily when six runs try it. With one run per task and configuration the two sources cannot be separated. The experiments with E4B below therefore use repeats.

7.4 Masked test runners

The harness has an option that masks the test runners inside the sandbox, so that the agent cannot run the tests of the repository. We did not know whether the hosted scorer sets it, so two arms were run on the first 30 tasks with the runners masked (Table 8). `BF5` solved 19 with masked runners against 17 without, and `SH5` 17 against 17. The standard and the masked runs come from different blocks. No drop was detected, and nothing in these counts suggests that the agents depend on running the tests.

7.5 Real hardware against emulated latency

Two blocks on a Kaggle notebook with four L4 GPUs repeated an arm on tasks that it had run under emulated latency (Table 9). Mean time per job agreed within 4 and 8%, with the emulated runs the slower ones, so the emulation did not make the time limit easier than it is on real hardware.

The solved counts moved in both directions. `SH5` solved 6 of 10 tasks on real hardware and

Table 8: Runs with the test runners masked inside the sandbox, on the first 30 development tasks (solved of 30).

Arm	Standard	Masked	Only masked	Only standard
`BF5`	17	19	2	0
`SH5`	17	17	2	2

Table 9: The same arm and tasks on real 4×L4 hardware (serial jobs) and under latency emulation (eight workers). Wall: mean seconds per job.

Arm	Tasks	Solved, L4	Solved, emul.	Both	Only L4	Only emul.	Wall, L4	Wall, emul.	Diff.
`SH5`	10	6	3	3	3	0	280	292	−4%
`SH6`	24	14	17	14	0	3	289	314	−8%

3 under emulation. These ten tasks had been picked as a hard mix, and three of them changed outcome with identical agent files. `SH6` solved 14 of 24 on real hardware and 17 under emulation. The 24 tasks of this second block were chosen with the earlier results of the arms in view, which favours the emulated count; a fall on repetition is what such a choice predicts, and it is no evidence of a difference between the machines. Neither block can support a claim about the solve rate. Together they show that a single run per task moves by several tasks in ten to twenty-four when nothing changes.

7.6 Berth tasks on real hardware

Two later blocks ran 31B arms on Berth tasks on four L4 GPUs with the software stack of the scorer. The first is a timing block with one job at a time (Table 10): the two public bundles and one variant of each on ten tasks. Every arm solved 4 of 10. The agent phase took 179 to 233 seconds on average, and grading about ten seconds.

The second block compared `BF5` with `BF5-c`, which adds a checking step before the patch is submitted, on 16 tasks with two repeats. `BF5-c` solved 13 of 32 runs and `BF5` 11; 2 tasks gained a solved run and 0 lost one, and the agent phase was 6% longer. No difference was detected.

7.7 Hosted submissions

Table 11 lists our five hosted submissions with the local result of the same arm. The scores are given as the leaderboard displays them. Two of them, 0.06 and 0.18, are not a count of 58 tasks rounded to two digits, so we do not convert scores into task counts.

Each score is one draw. The unchanged `BF5` files scored between 8 and 11 of 58 in ten public draws [7], and our draw of these files, 0.18, lies in that range (Figure 6). `W3` scored 0.12 and `BF5` 0.18, while locally they solved 38 and 37 of the same 70 tasks. With a standard deviation near 1.5 tasks for one draw, the difference of two draws of the same files has a standard deviation near 0.04. The hosted difference of 0.06 is about one and a half times that: it does not contradict the local tie, and it does not confirm it. This is the situation that the introduction describes: a difference of the size that development produces is not visible in a hosted score.

The two E4B files scored 0.06 (`base`) and 0.05 (`v-a3`, the third candidate of Section 10). No repeated draws of an E4B file exist, so the noise of an E4B score has not been measured. We read no direction from a difference of 0.01 between two single draws. The local confirmation run of the same candidate (Section 10.2) is the evidence that we have, and its verdict is “not confirmed”.

Table 10: Timing block: four 31B arms on ten Berth tasks, jobs run one after another on 4×L4 with the software stack of the scorer. Times are mean seconds per job.

Arm	Solved	Agent	Setup	Verify	Tokens out	Tool calls
`BF5`	4/10	191	2.9	10.5	6,454	18.5
`BF5-s`	4/10	233	4.8	10.1	7,710	19.8
`ID24`	4/10	193	3.8	10.7	6,551	16.7
`ID24-r`	4/10	179	4.0	7.8	5,792	17.1

Table 11: Our five hosted submissions (one per day) and the local result of the same configuration. The public score is the solved share on the public part of the hidden test set, as displayed.

Date (UTC)	Arm	Model	Public score	Local result of the same arm
2026-11-05	`W3`	31B	0.12	38 of 70 development tasks (emulated latency)
2026-11-08	`ID24`	31B	0.17	4 of 10 Berth tasks (4×L4)
2026-11-09	`base`	E4B	0.06	20.2% of 1,616 Berth runs (workstation)
2026-11-10	`BF5`	31B	0.18	37 of 70 development tasks (emulated latency)
2026-11-11	`v-a3`	E4B	0.05	118 of 512 Berth runs (panel H)

The local solve rates are far above the hosted scores for both models, on the development tasks and on Berth. Local counts compare arms on the same tasks. They do not predict a hosted score.

8 Experiments with E4B

E4B generates 2.8 times as many tokens per second as the 31B model on the scorer's hardware (Section 3), and a separate prize of the competition is reserved for the small models. Most runs of this paper, 4,314 of 5,024, are E4B runs. This section reports what we measured before any candidate change was tested: the first runs on hosted notebooks, the controls of the grading path, the agreement between the workstation and the hosted notebook, how far the unchanged baseline repeats, where it succeeds and fails, and what a run costs. The three programmes that tried to improve the baseline follow in Section 10.

8.1 First experiments on hosted notebooks

Development tasks. The first block (`pubE4-1`, Table 12) ran three E4B arms on 24 development tasks with small reference fixes, from three of the four repositories: a single agent (`e-single`), the wrapped design of Section 7.1 (`e-wrap`), and the wrapped design with a fine-tuned adapter on the coder (`e-wrap-ad`). Each arm solved 5 of the 24 tasks. The arms largely solved the same tasks: 4 tasks were solved by all three and 6 by at least one. On these 24 tasks the 31B arms `BF5` and `SH6` had solved 16 and 19, each in one run per task on the emulated-latency machine. The block therefore shows the size of the gap between the two models on tasks chosen to be easy, and no difference between a single agent, a wrapped agent and an adapter. With one run per task and five solved tasks per arm, it could only have shown a very large difference. A job took 117 s on average and used about 10 tool calls: E4B stops by itself, far inside every limit of the scorer.

Berth tasks. The second block (`berthE4-k1`) is the first on Berth tasks: 24 tasks, two repeats, and two arms in one job queue. `e-BF` is the public single-agent prompt adapted to E4B; `base` is the withheld E4B agent bundle that we later submitted. `base` solved 8 of 48 runs and `e-BF` 4 of

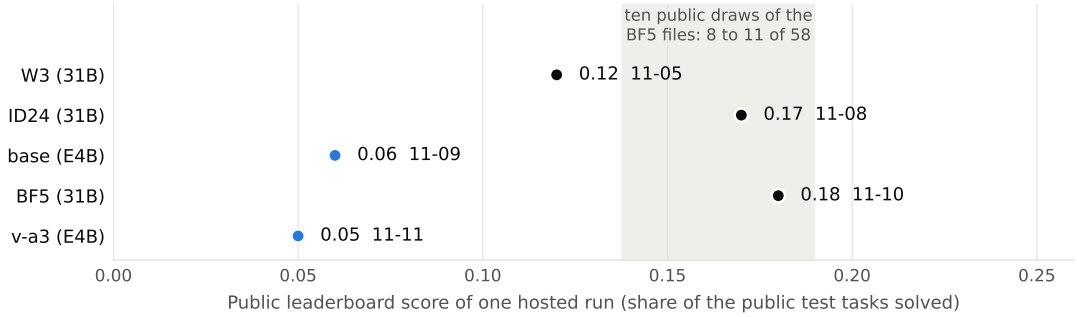


Figure 6: Our hosted scores, one run each, against the range of ten public draws of the unchanged `BF5` files [7].

Table 12: First E4B experiments on hosted 4×L4 notebooks. Time: mean seconds per job (whole job in the first block, agent phase in the others).

Block	Arm	Tasks	Runs	Solved	%	95% interval	Time	Tool calls
`pubE4-1`	`e-single`	24	24	5	21	9–40	124	8.7
`pubE4-1`	`e-wrap`	24	24	5	21	9–40	108	10.3
`pubE4-1`	`e-wrap-ad`	24	24	5	21	9–40	118	10.5
`berthE4-k1`	`e-BF`	24	48	4	8	3–20	70	9.3
`berthE4-k1`	`base`	24	48	8	17	9–30	79	11.9
`berthE4-k2`	`base`	8	16	7	44	23–67	82	12.4
`berthE4-k2`	`base-p`	8	16	6	38	18–61	80	12.1
`berthE4-k2`	`e-BF`	8	16	5	31	14–56	58	8.9
`berthE4-k2`	`e-BF-s`	8	16	2	12	3–36	60	9.3

48. By task, `base` was ahead on 4 tasks and behind on 1 (sign test, $p = 0.38$); no difference was detected. We kept `base` as the baseline of every later comparison. This block is not evidence that it is the better of the two; it is the bundle we meant to submit, and it was not behind.

The third block (`berthE4-k2`) compared four arms on eight tasks, one from each of 8 repositories, with two repeats: `base`, `base` with one prompt change (`base-p`), `e-BF`, and `e-BF` with changed sampling settings (`e-BF-s`). They solved 7, 6, 5 and 2 of 16 runs. The intervals of Table 12 overlap widely, and we drew one conclusion only: 16 runs per arm on a hosted notebook cannot rank prompt variants of this model. The eight tasks share no task with the 24 of the second block, and the record does not say how they were picked. `base` solved 44% of its runs on them, twice its share elsewhere, so the block says nothing about the level of any arm. Neither `base-p` nor `e-BF-s` was run again.

These three blocks hold 232 runs. All later E4B experiments ran on the workstation, where the same number of runs takes about an hour.

8.2 Task pool and grading controls

A solved share means something only if the grading path separates a correct fix from no fix. Before any agent ran on a task, the task was graded twice: once with its reference patch, which must pass, and once with no patch, which must fail (Table 13). On the workstation, without a container, the reference patch passed on all 580 tasks that loaded, from 24 repositories, and the empty patch passed on 1; that task was removed, which leaves the 579 tasks of Section 2. The controls were then

Table 13: Control runs of the grading path: every task graded once with its reference patch (must pass) and once with no patch (must fail).

Tasks and machine	Reference patch passes	Empty patch passes
whole pool, workstation, no container	580/580	1/580
panels C and H, final runner	200/200	0/200
screen set S, final runner	48/48	0/48
case-study panel and two tasks of S, final runner	233/233	0/233
hosted notebook, first batch	38/38	0/38
hosted notebook, second batch	87/87	0/209

Table 14: The baseline on the same 24 Berth tasks, one run per task, on the hosted notebook and on the workstation under successive runners. Agent: mean seconds of the agent phase.

Run	Solved	95% interval, %	Agent
hosted 4×L4, repeat 1	4/24	7–36	79
hosted 4×L4, repeat 2	4/24	7–36	80
workstation, no sandbox, 8 workers	3/24	4–31	65
workstation, no sandbox, 1 worker	5/24	9–40	63
workstation, container runner, version 1	3/24	4–31	69
workstation, container runner, version 3	2/24	2–26	58
workstation, container runner, version 4, run a	4/24	7–36	83
workstation, container runner, version 4, run b	3/24	4–31	70
workstation, container runner, version 7 (final)	5/24	9–40	71

A further workstation run under runner version 2 solved 4 of 24; its result rows were not kept and it is not in the released file.

repeated under the final container runner for every panel used in this paper, and on CPU notebooks of the hosting platform for 107 tasks with the reference patch and 209 with the empty patch. Every one of these controls gave the expected result. 1,016 pairs of task and control were graded on more than one machine or runner, and 0 of them changed its result. The file `controls.csv` holds all 2,494 rows.

The controls test the grader, not the agent's environment. They would not show a difference in what the agent sees while it works, for example a different Python version in its shell; Section 8.3 addresses that side.

8.3 Workstation against the hosted notebook

The workstation serves the model from one GPU instead of four, runs jobs in parallel, and puts each job in a container. Any of these could change what the agent does. Table 14 lists every run of the baseline on the same 24 Berth tasks, one run per task: two repeats on the hosted notebook with four L4 GPUs, and seven runs on the workstation as the runner changed. The first two workstation runs used no sandbox and still had the role divergence described in Section 1; the others used the container runner in four of its versions.

The hosted repeats solved 4 and 4 of 24. The workstation runs solved between 2 and 5, in total 25 of 168 runs (14.9%) against 8 of 48 (16.7%) on the hosted notebook. Over all 216 runs only six distinct tasks were ever solved, so the totals move because the same few tasks come out or not, not because a runner opens or closes other tasks. The mean agent phase took 79 and 80 s on the hosted notebook and 58 to 83 s on the workstation. Figure 10A shows the difference of each workstation

Table 15: The unchanged baseline run again on the same tasks in separate sessions on the workstation. Range: largest minus smallest session total in percentage points. Same: tasks with the same solved count in every session. Never, always: tasks solved in no run, in every run. r: correlation of the per-task counts of the first and last session.

Task set	Sessions	Solved runs	%	Range	Same	Never	Always	r
S (48 tasks, 2 repeats)	3	19, 21, 20	19.8, 21.9, 20.8	2.1	39	34	5	0.91
H (128 tasks, 4 repeats)	2	103, 97	20.1, 18.9	1.2	99	83	10	0.92

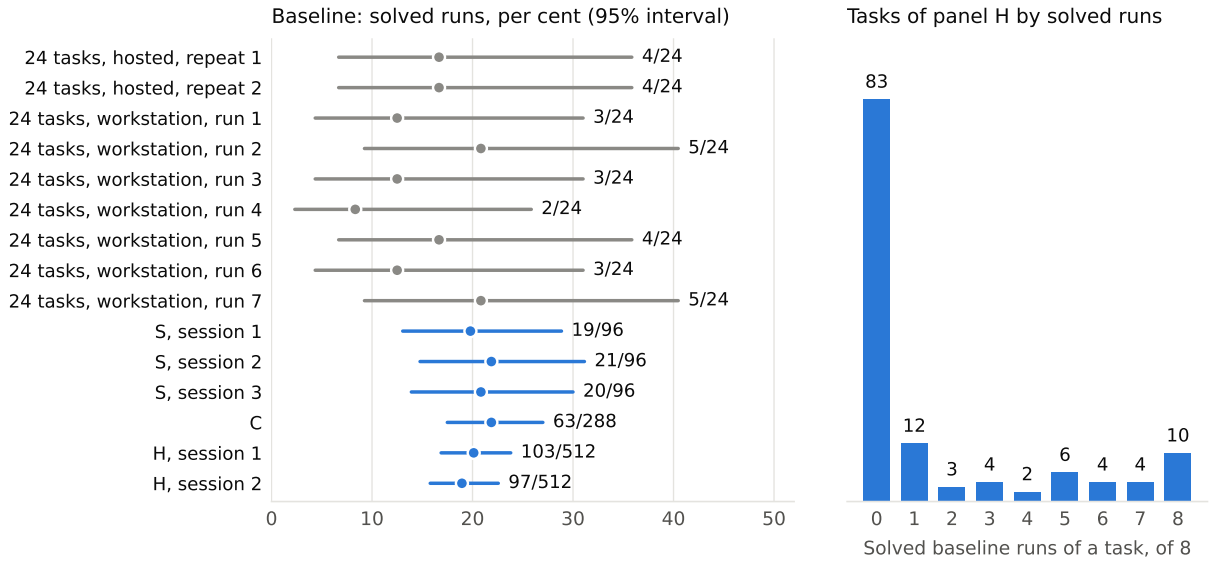


Figure 7: The unchanged E4B baseline on the workstation. Left: solved runs of each session on the same tasks, with 95% intervals. Right: the 128 tasks of panel H by the number of baseline runs that solved them, of eight runs in two sessions.

run from the hosted share; it includes an eighth run, under runner version 2, whose rows were not kept.

What this supports is limited. With 24 tasks and one run each, the 95% interval of 4 of 24 runs from 7% to 36%. The table rules out a gross difference between the machines, such as the workstation solving half or double the hosted share. It cannot show that a specific difference in the prompts has no effect. That question needs an experiment with power, and Section 9 reports one.

8.4 How far the baseline repeats

Every screen and every confirmation block ran the baseline in the same job queue as the candidates. The baseline therefore ran three times on the 48 tasks of S and twice on the 128 tasks of H, in separate sessions (Table 15 and Figure 7). On S it solved 19, 21 and 20 of 96 runs; on H, 103 and 97 of 512. The session totals differ by at most 2.1 points on S and by 1.17 points on H.

Repeats of one task can differ. They rarely do. On H, 83 of the 128 tasks (65%) were solved in none of the eight baseline runs and 10 in all eight; only 35 tasks were solved in some runs and not in others. The per-task solved counts of the two sessions correlate at 0.92. Because so few tasks are uncertain, the run-to-run standard deviation of the total is small. Estimated from the variation within tasks, it is 5.2 runs of 512, or 1.01 points, for one arm and 1.44 points for the

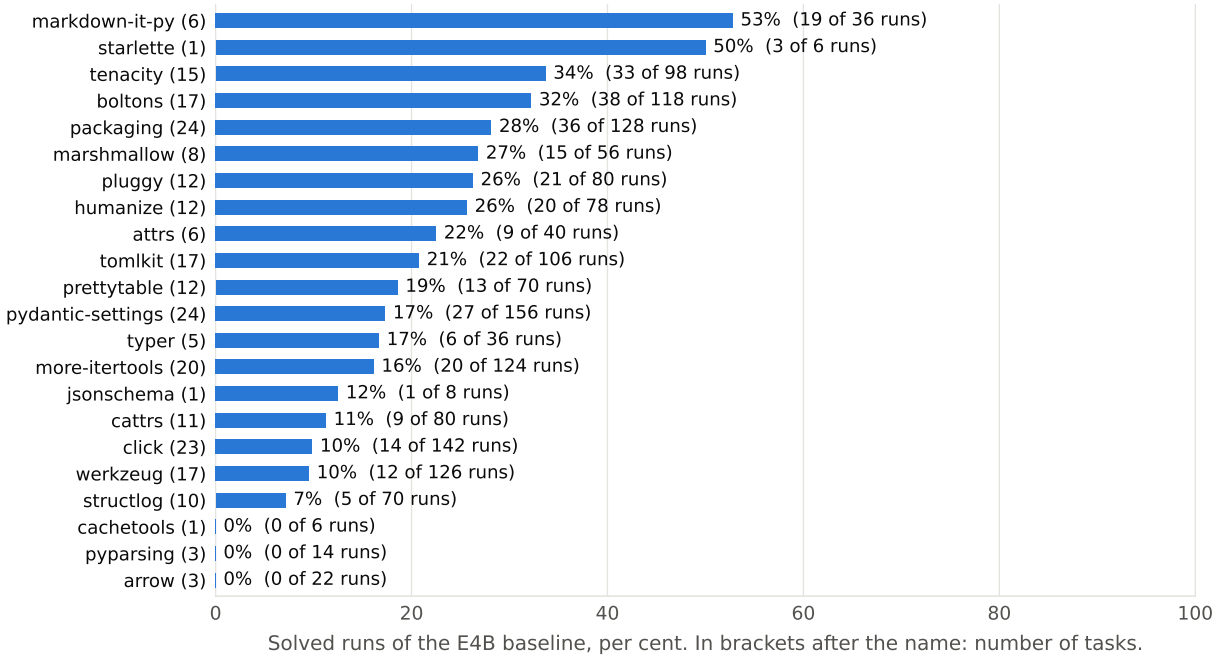


Figure 8: Solved share of the E4B baseline by repository, over all its runs in the screen and confirmation blocks, with 95% intervals that treat runs as independent. Numbers of tasks and runs are in Table 16.

difference of two arms on the same tasks. The two baseline sessions differ by 6 runs, which is 0.8 of that standard deviation; the one pair of repeats that we have does not contradict the estimate. The binomial formula, which treats the 512 runs as independent with one common probability of 19.5%, gives 1.75 points; the variation within tasks accounts for 34% of that variance. Section 11 uses these numbers.

8.5 Repositories

Over the screens and confirmations, the baseline ran 1,600 times on 248 tasks from 22 repositories and solved 323 runs (20.2%). Table 16 and Figure 8 split this by repository. The share runs from 53% in the best repository to zero in 3 repositories; the three repositories with the most solved runs hold 33% of all solved runs. The zero rows have one to three tasks each, so they say little about those repositories. The wider pattern is firm, though: with 10 to 24 tasks each, the shares of the larger repositories still range from 7% to 34%.

Two consequences follow. A change that helps in one repository can move the total visibly while doing nothing elsewhere, which is why the pass rule of Section 6 counts repositories and tests with repositories as clusters. And a local total is a property of the repository mix as much as of the agent: the hidden tasks of the scorer need not come from these repositories, and nothing here says how the baseline does on them.

8.6 Time, tokens and failure kinds

Table 17 gives time, tokens and endings for every setting in which agent runs were made, and Figure 9 the distribution of time for the two largest groups.

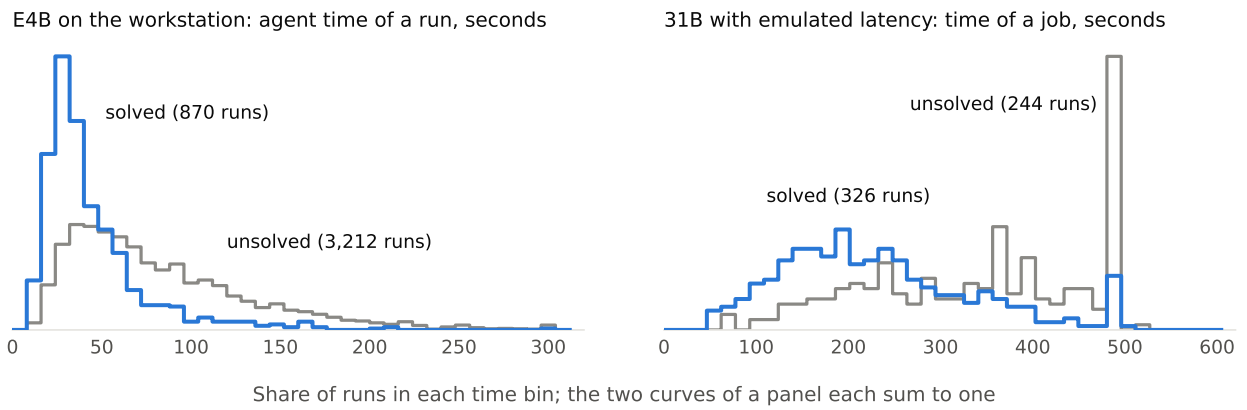


Figure 9: Time of solved and unsolved runs for the two models. E4B: agent phase of the Berth runs on the workstation. 31B: whole job of the development-task runs under emulated latency.

E4B is quick, and quicker when it succeeds. On the workstation the agent phase of an E4B run took 75 s on average: 45 s when the task was solved and 84 s when it was not. The 31B runs on development tasks took 278 s, with 231 s and 341 s. The time limit ended 12.6% of the 31B runs and 0.3% of the E4B runs. For the 31B model the limit is a working constraint, and the arms of Section 7 differ mostly in how they spend it. For E4B it is not: more time would change almost nothing.

E4B fails by handing in a wrong fix. Of the 3,212 unsolved E4B runs on the workstation, 97% ended because the agent declared its work finished. 93% handed in a patch that the tests rejected, 7% left no patch, and 0.4% ran into the time limit. The typical failure is thus a wrong fix handed in after about a minute and a half, not a run that was cut off. The three programmes of Section 10 accordingly changed what the agent does within a run (its prompt, the way it edits files, an added agent step) and left the limits alone, with one exception, an arm with a changed thinking-token limit.

Both models on the same tasks. 26 Berth tasks, all in S, were run with both models: with five 31B arms, the public configurations and variants of them, on four L4 GPUs (Section 7), and with the E4B baseline in the three screens on the workstation. The 31B arms solved 40 of 104 runs (38.5%) and the E4B baseline 20 of 156 (12.8%). The 31B arms solved 11 of the tasks at least once and E4B 5; no task was solved by E4B only. An average run took 226 s against 64 s, so the agent time spent per solved run is of the same order: 589 s for the 31B arms and 496 s for E4B. The comparison mixes machines and arms and covers 26 tasks; we read it as a description, not as a test. It does say that the small model is not cheaper per solved task. It is cheaper per attempt, and it reaches fewer tasks.

Totals. The 4,082 E4B runs on the workstation took 85 hours of agent time in total, spread over parallel workers, and used 637 million prompt tokens and 33 million generated tokens.

9 Case study: four prompt divergences on E4B

Section 5 lists ways in which a local stack can send the model other prompts than the scorer does, and Section 8.3 shows that a comparison of 24 tasks cannot say whether such a divergence matters. This section reports the experiment that can. It takes four divergences that we or public notebooks had actually run with, turns each into one arm, and measures E4B's solved share against the scorer's path on 231 tasks with four repeats. The design, the analysis and the power simulation were registered before the first job.

Section 9.1 is the account of the short paper. The subsections after it give the full record: how the block ran, every outcome count, the primary and secondary analyses with their sensitivity checks, how many requests each divergence actually changed, and the exploratory measures of time and tokens. The rows are in the released files `e8_runs.csv`, `e8_tasks.csv` and `e8_repositories.csv`, and Appendix B repeats the analysis with the released kit.

9.1 Design and main result

Five arms share one model server, one withheld agent bundle, google-adk 1.36.1 and all sampling settings.

No newline arm was run: both templates render it identically on this server.

Panel. Removing tasks used by other panels, reserved repositories, slow tasks and duplicates from the 579 leaves 231 tasks in 15 repositories. They run 4 repeats in 5 arms (4,620 jobs); for each task and repeat the arms run back to back in counterbalanced order.

Analysis, frozen before any outcome (protocol sha256 `070c9655d96372d054f430727259de7b25af2ca93c3ec4c9295dfdc37aeb8cf4`). The estimand d_X is the mean over tasks of arm X's solve rate minus arm F's. The primary test is an exact two-sided task-level sign-flip test, Holm-corrected over the four arms at 0.05; only then do we say that X raises or lowers E4B's solve rate *on these tasks*. Intervals (unadjusted) invert that test. A repository-cluster bootstrap interval is secondary (lower simulated power). A frozen checker re-renders every returned request from the job's dump, matches the server's token count and confirms the job's arm; a failing job is re-run. Outcomes stayed blind until a pre-registered cut-off (2026-11-11 06:00 UTC) and were unblinded once.

Power, simulated before data. The family-wise false-claim rate is 1.2–4.2%. At 231 tasks an even 5-point drop in one arm is detected with probability 82–100% across the simulated settings; the drop or gain detected with 80% power ranges from below 3 to about 5.2 points.

Deviations recorded blind. Independent validation of the final runner finished only after launch. From about an hour in, a second workload held about 9.6 GiB of GPU memory, so timings are under mixed load. The block stopped three times: after 967 jobs at the runner's open-file limit (cause unknown), after 378 more by an external termination signal on the shared machine, and after 3,074 more when the kernel's out-of-memory killer ended the model server (cause undiagnosed). Each resume ran only never-started jobs; the 24 interrupted jobs were re-run once.

Results. All jobs finished before the cut-off. Every arm has a valid row in 924 of 924 slots, and the checker passed all 4,620 analysed jobs. No pre-emption was recorded; 6.0% of runs took over 240 s, above the pre-registered 5% threshold. No difference from F was detected for any arm (Table 18, Figure 10B; smallest Holm-adjusted p 0.44); the secondary repository-cluster intervals

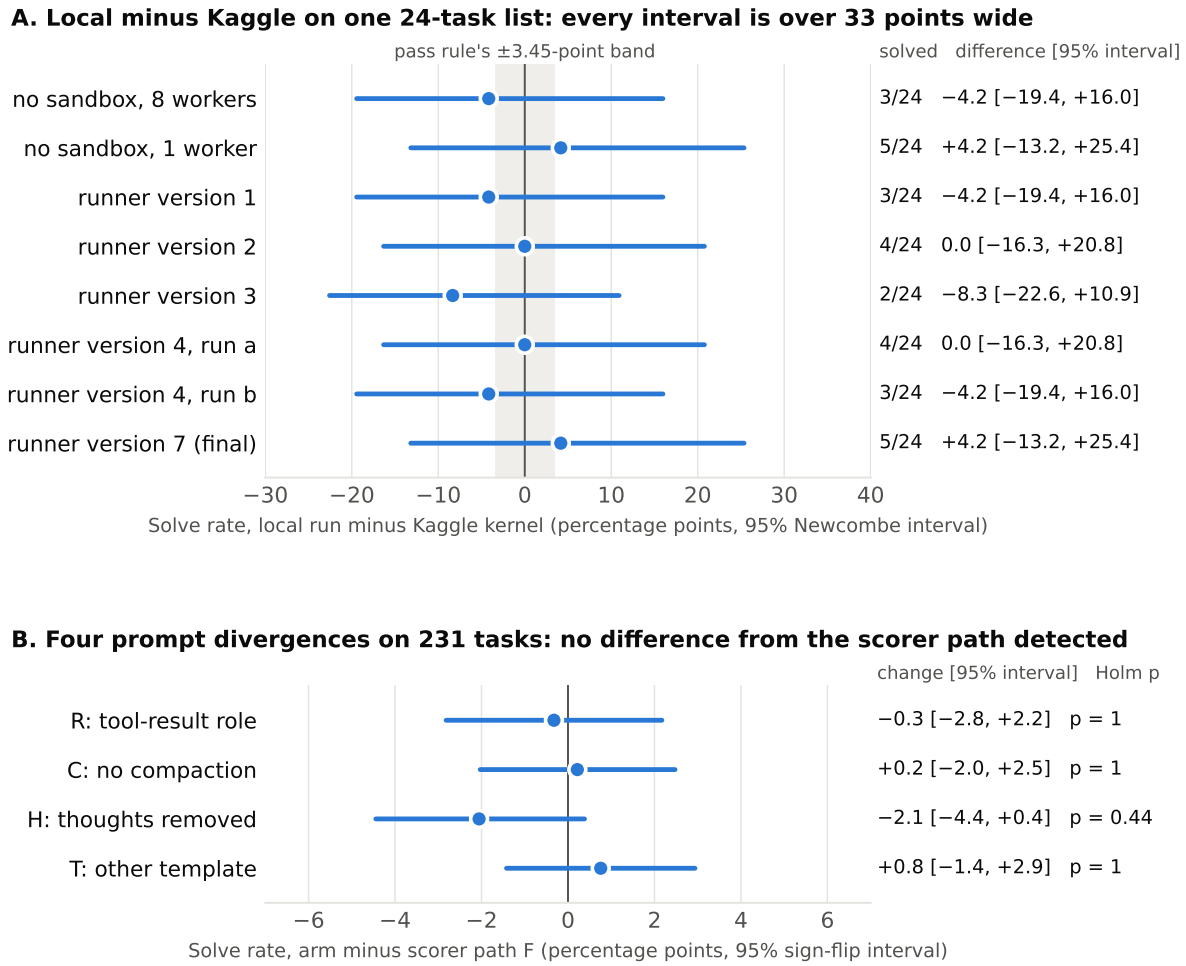


Figure 10: (A) Solve-rate difference, local minus Kaggle (pooled 8/48), for eight local runs of one 24-task list (95% Newcombe intervals, runs treated as independent); shaded: the pass rule's ±3.45 points. (B) Change in solve rate against F on 231 tasks: estimates, 95% intervals and Holm-adjusted p.

all include zero. The largest estimate is H's −2.1 points (unadjusted p 0.11; −1.3 with time-limit rows as missing). In exploratory measures, H runs averaged 137 s against 78 s and ended unsolved at the bundle's five-minute limit in 11.5% of runs against 1.8%; in C, 12.3% of runs ended unsolved on a rejected over-long prompt. These are null results, not evidence of no effect: the simultaneous 98.75% intervals allow a drop of 5.1 points (H) and a gain of 3.5 (T).

9.2 Execution record

The first job started on 2026-11-10 06:39 UTC and the last one ended on 2026-11-11 03:56 UTC, before the cut-off (2026-11-11 06:00 UTC). The block ran as four batches and one batch of re-runs (Table 19). Three batches stopped early, for the reasons in the last column, and each following batch started only jobs that had never started. In all, 4,644 jobs were started: the 4,620 analysed runs and 24 interrupted jobs. Of the interrupted jobs, 16 left an infrastructure row; the other 8, all in batch 1, left none and are counted from the list of started jobs. Each interrupted slot was re-run once, so every slot has exactly one analysed run. The released file holds every written row.

276 of the 4,620 runs (6.0%) took more than 240 s of agent time. This is above the 5% share that the protocol set as the sign of a loaded machine: from about an hour into the block a second workload held part of the memory of the GPU (Section 9.1), so the times are under mixed load. No pre-emption was recorded.

9.3 Outcomes and solve rates

Table 20 splits the runs of each arm by how they ended and Table 21 gives the solve rates, which lie between 21.8% and 24.6%. Two arms end differently from F. In C, 114 runs (12.3%) ended unsolved because a prompt grew past the server's limit and was rejected; no F run ended this way, and 48% of the F runs made at least one history summary. In H, 106 runs (11.5%) were unsolved at the bundle's five-minute limit, against 17 (1.8%) in F.

Table 22 shows why a difference between arms is hard to detect on this panel: 135 of the 231 tasks were solved in no run of any arm and 13 in every run, so the other 83 tasks carry every difference.

9.4 Primary analysis: tasks

For a task, let K be the number of solved repeats in the arm minus the number in F, from -4 to $+4$. The estimate d is the mean of $K/4$ over the 231 tasks. Under the null hypothesis the arm and F are exchangeable within a task, so each K is as likely to carry one sign as the other; the test compares $|\sum K|$ with its distribution over all sign assignments, computed exactly. Holm's method adjusts the four p-values. The intervals invert the same test under a constant shift, with 99,999 frozen Monte Carlo sign draws.

No difference from F was detected for any arm (Table 23, Figure 11). In every arm between 172 and 183 of the 231 tasks have the same number of solved repeats as in F. The largest estimate is that of H: 34 tasks have fewer solved repeats than in F and 20 have more, which gives -2.06 points with exact $p = 0.11$ and Holm-adjusted $p = 0.44$. These are null results, not evidence of no effect. The simultaneous 98.75% intervals show what the experiment cannot exclude: a drop of up to 5.1 points (H) and a gain of up to 3.5 points (T).

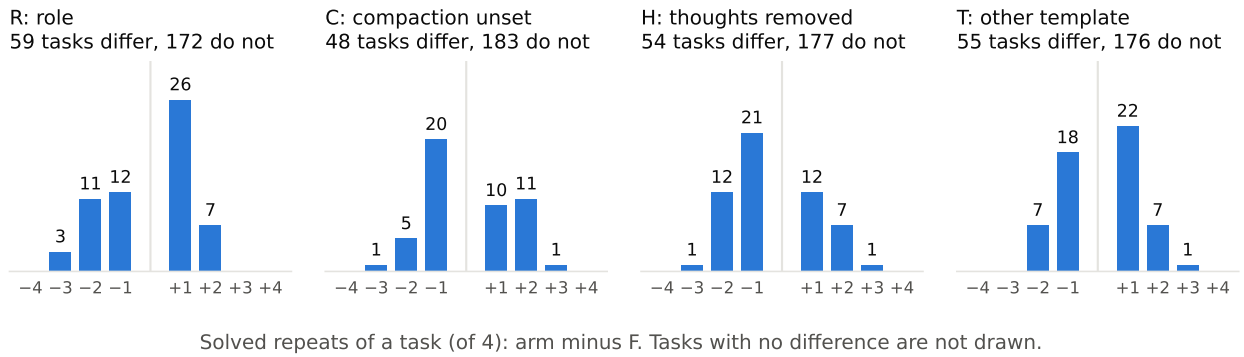


Figure 11: Tasks whose number of solved repeats differs between an arm and F. Bars to the left of the centre line are tasks with fewer solved repeats in the arm.

9.5 Secondary analysis: repositories

Tasks of one repository tend to succeed and fail together, so the registered secondary analysis takes the 15 repositories as its units (Table 24). Table 25 and Figure 12 show each repository. All four repository-level intervals include zero, and the smallest two-sided p is 0.18 (H). The leave-one-repository-out estimates of H stay negative (-3.01 to -1.44 points), so no single repository produces the estimate of H; those of R, C and T change sign.

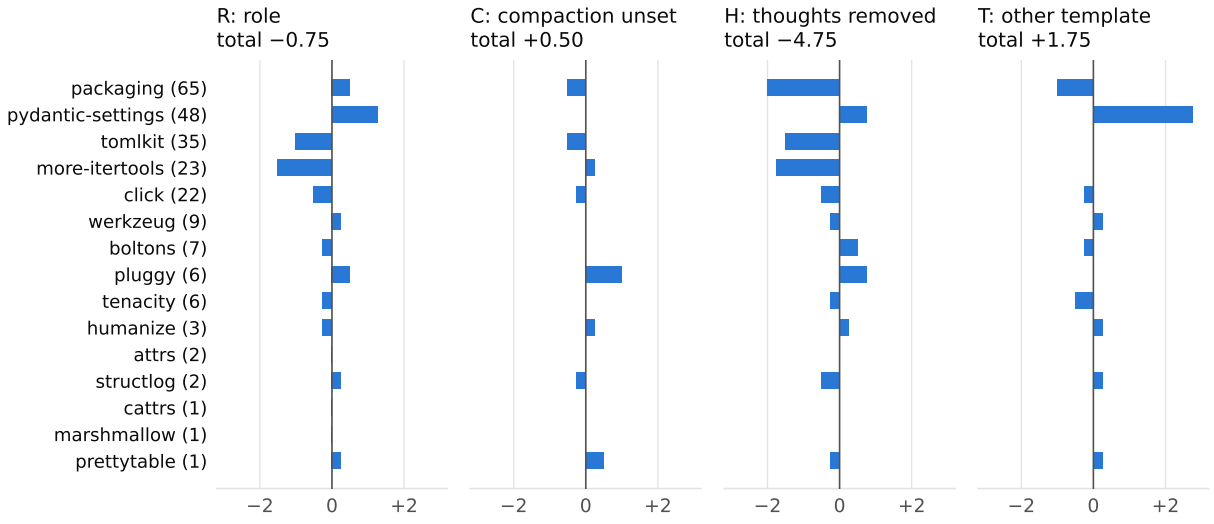


Figure 12: Change against F in each repository, per arm. Repositories are ordered by their number of tasks.

9.6 Sensitivity analyses

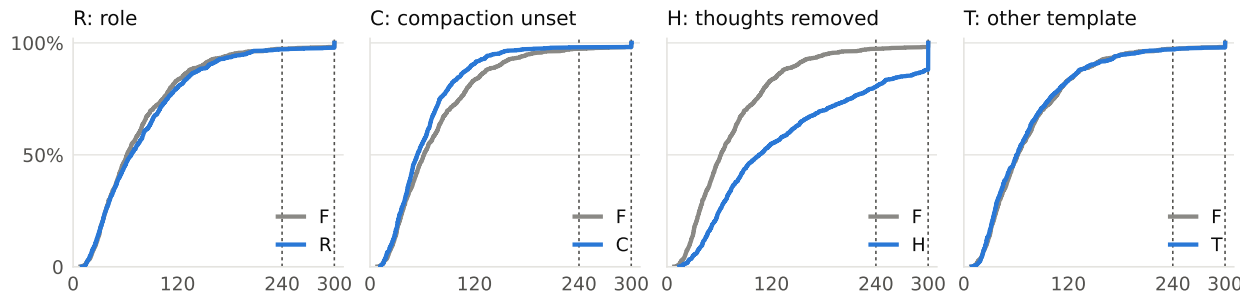
Three sensitivity analyses were registered with the primary one (Table 26). The first treats the runs that were unsolved at the five-minute limit as missing, in both arms; the estimate of H is then -1.25 points ($p = 0.34$) instead of -2.06. The other two score as unsolved every slot whose first job was interrupted, in the arm or in F. No sensitivity analysis gives a p-value below 0.05.

9.7 Exposure and the prompt check

A divergence can only matter in the requests it changes. The frozen checker re-rendered all 75,606 returned requests from the request dumps of the jobs. For each of the 4,620 analysed runs it confirmed that the prompts carry the divergence of the run's arm and no other difference; 0 runs failed the check. The divergence changed 85 to 90% of the requests in R, H and T, and 21% in C (Table 27).

9.8 Exploratory run measures

Table 28 compares thirteen measures of a run between each arm and F. These comparisons were not part of the registered tests and are not adjusted for multiplicity. H runs are longer: 137 s of agent time on average against 78 s in F (medians 106 and 62 s; Figure 13), with 14,397 completion tokens per run against 7,690.



Agent time (s). Dashed: 240 s (load threshold) and 300 s (the bundle's limit). Vertical axis: share of runs finished.

Figure 13: Agent time per run: each arm (blue) against F (grey).

9.9 What the case study does and does not show

The four divergences are not small edits. Three of them change between 85% and 90% of all requests of a run, and one removes the model's earlier reasoning from every request. Yet the simultaneous intervals exclude a loss of more than 5.1 points and a gain of more than 3.5 points for any of them. For E4B on these tasks, the solved share is insensitive to prompt details that we had expected to matter, and that public discussion treats as traps.

Three limits apply. First, the result holds for this model, this agent bundle and these 15 repositories; we did not run the experiment with the 31B model. Second, the arm without history thoughts shows the largest estimate, takes almost twice as long per run and ends at the time limit more often. Its estimate therefore depends on the time limit: with the time-limit rows treated as missing it is -1.3 instead of -2.1 points. Third, an insensitive solved share does not make conformance checks useless. A check is cheap, and it removes a question that otherwise needs 4,620 jobs to answer.

The case study also calibrates the rest of this paper. Its reference arm solved 23.8% of its runs, and with 924 runs per arm the 95% interval of each difference reaches 2.2 to 2.5 points to either side of the estimate. A difference between two E4B arms that is to be seen needs this order of size, and Section 10 shows what happens with fewer runs.

10 Three attempts to improve the E4B baseline

With a runner that could be trusted and a baseline that repeats, we tried three times to raise E4B's solved share. The three programmes changed different things. The first changed the prompt; one of its variants changed a thinking-token limit instead. The second changed the way the agent edits files. The third added a sub-agent step. What exactly was changed is withheld with the bundle until the end of the competition (Section 2); the arms are named in Table 32, and every run is in `runs.csv`.

Each programme followed the same four steps.

1. *Build* several variants of the baseline bundle.
2. *Pilot* (programmes 2 and 3 only): run first builds on 15 or 16 tasks, one run each, to find defects.
3. *Screen*: run the variants and the unchanged baseline in one job queue on the 48 tasks of S, two repeats each, and take the variant with the most solved runs as the candidate.
4. *Confirm*: register the candidate, the tasks, the rule and the analysis script, then run candidate

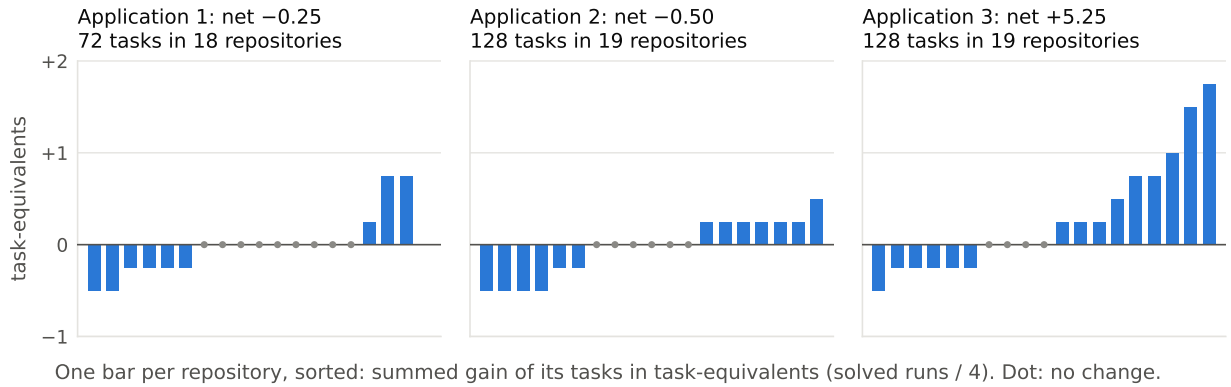


Figure 14: Gain of each repository in the three applications of the pass rule (candidate minus baseline). Repositories are sorted by their total within each application.

and baseline once on a frozen panel that shares no task with S, and apply the pass rule of Section 6.

The whole sequence for one programme took 1,152 to 1,402 runs and three to five hours of workstation time.

10.1 Pilots and screens

Table 29 lists every pilot and screen block.

Pilots. Programme 2 began with two first builds on 15 tasks of the case-study panel (`pilot-2a`), which solved 4 and 5 of 15, and continued with four rebuilds on the same tasks (`pilot-2b`), which solved 3, 7, 6 and 6. No baseline ran in these two blocks. Of the four rebuilds, `v-t4` and `v-t6` went on to the screen. Programme 3 began with two first builds on 16 tasks of the small panel C (`dev-3`); they solved 2 and 3 of 16 against 4 for the baseline in the same queue, and both were rebuilt before the screen. The 16 tasks of `dev-3` belong to the panel of the first confirmation, which had ended by then, and to no later one.

The pilots were meant to find builds that did not work. They ranked nothing. `v-t4` had the best pilot result, 7 of 15, and then solved fewer runs than the baseline in its screen.

Screens. In the first screen all five variants solved more runs than the baseline, 22 to 25 of 96 against 19, and the leader, `v-p2`, led by +6.2 points. In the second screen, `v-t6` led by +1.0 points and `v-t4` trailed. In the third, `v-a3` led by +5.2 points and `v-a4` trailed. All three leaders used more agent time than the baseline of their screen: 75 against 64 s, 89 against 72 s and 77 against 66 s.

10.2 Confirmation runs

The pass rule of Section 6 was applied three times to a real candidate: a changed version of the withheld agent bundle, tested against the unchanged baseline on the same tasks and repeats. Each application ran once. Its rule, tasks and analysis script were frozen before its first job, and the outcomes stayed unread until the run had ended. Table 30 gives every quantity that the rule uses, and Figure 14 the gain of every repository.

How the candidates were chosen. Each candidate was the leader of a screen on one separate 48-task set, with two repeats per task: 576 runs over the baseline and five variants for the first candidate, and 288 runs over the baseline and two variants for each of the other two. Each leader solved more runs than the baseline in its screen (25, 22 and 25 of 96, against 19, 21 and 20). A screen selects a candidate and tests nothing, because the leader of several variants is expected to look better in its screen than it is. No task of the screens is among the tasks of an application. Two small pilots came before the screens of the later candidates. The variants of the second candidate were first tried on 15 tasks of the case-study panel, which shares no task with an application. Two predecessors of the third candidate were tried on 16 tasks of the small panel, that is, on tasks of the first application and on none of the second and third.

Tasks and conditions. The first application used the small panel (72 tasks) and all five conditions of Section 6. The second used the 128 tasks of the large panel that are outside the small one. The third used the same 128 tasks as the second, and both arms ran again. Its registration stated three things before the run: that this was the second use of that task set, that the candidate had run on none of these tasks, and that the candidate was built after the result of the second application was known. It also named two mechanical reads of earlier runs that went into a repair of the candidate and that may include runs on these tasks: a scan of the agent's commands in the 1,024 runs of the second application, and the output of one command in six earlier runs. Neither read used whether a task was solved. The candidate had been submitted to the competition once before the run; its hosted score had not arrived when the outcomes were opened, and it is not an input of the rule. The second and third applications registered conditions (1) to (4) only.

Results. The rule asks for a mean gain of at least 3.45 points with a one-sided p of at most 0.025, and for further conditions on repositories, time and infrastructure errors. No candidate passed. The first two estimates are below zero: -0.35 points (62 against 63 solved runs of 288; $p = 0.60$) and -0.39 points (101 against 103 of 512; $p = 0.60$). The third candidate solved 21 runs more than the baseline (118 against 97 of 512), a gain of $+4.10$ points. This gain is above the required size, and conditions (3) and (4) are met: 9 of the 19 repositories gain, 6 lose, and the lowest leave-one-repository-out gain is $+2.94$ points. But the one-sided p is 0.04, above 0.025, so condition (2) is not met and the decision is FAIL.

How to read the decisions. The verdict of a failed application is "not confirmed". It is not evidence that the candidate has no effect: the registration of the third application estimated, before the run, that a run of this size detects a true gain of the required size about half of the time. Nor do we call the third candidate an improvement. Its p is not corrected for the three applications, and it is also above 0.0083, the level divided by three. Its gain is uneven: the three repositories with the largest totals hold 27 of the 128 tasks and gain $+4.25$ task-equivalents, while the net gain over all repositories is $+5.25$ (Figure 14). A gain concentrated in a few repositories is the case in which the rule has little power (Table 5), and it is also what a few repositories can produce by chance. The first two leaders did not keep their screen lead on new tasks; this is consistent with the selection effect that the rule exists to catch.

10.3 What a screen lead is worth

The three candidates led their screens by $+6.2$, $+1.0$ and $+5.2$ points. On new tasks their gains were -0.35, -0.39 and $+4.10$ points (Figure 15). Two leads vanished; the third candidate kept most of its lead and still did not pass. We ask here, after the fact, what a lead on S is worth.

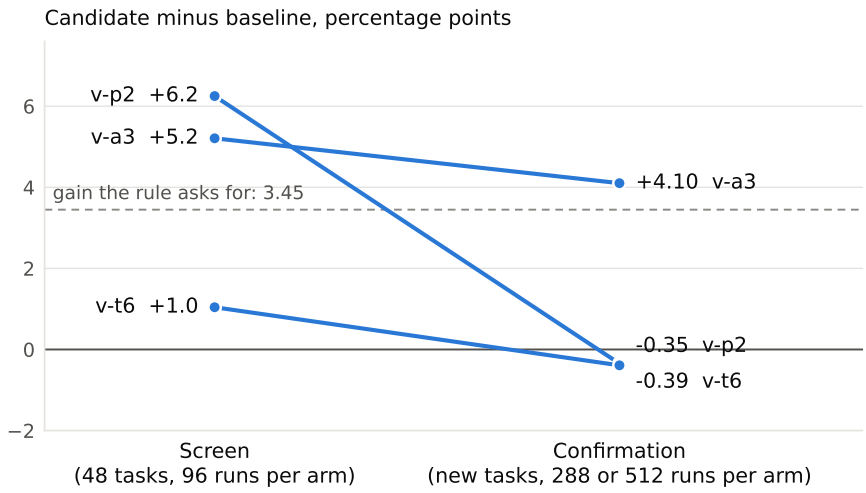


Figure 15: Lead of each candidate in its screen on S against its gain in the confirmation run on other tasks, both in percentage points against the baseline of the same block. The horizontal line is the gain that the pass rule asks for.

The noise of a screen. The baseline ran in all three screens, so each task of S has six baseline runs. The variation within tasks gives a standard deviation of 2.0 runs, or 2.1 points, for the total of one arm of 96 runs, and 2.9 points for the difference of two arms. S is a hard set for this model: 34 of its 48 tasks were solved in none of the six baseline runs, so the whole signal of a screen sits on 14 tasks.

The leader of equal arms. Suppose that no variant differs from the baseline, and that every arm total varies independently with the standard deviation above. Then the best of five variants leads the baseline by 2.4 points on average, and the best of two by 1.2 points. A lead as large as the one observed, or larger, has probability 0.09 in the first screen (6 runs, five variants), 0.59 in the second (1 run, two variants) and 0.10 in the third (5 runs, two variants). None of the three leads was unusual for a screen in which nothing differs.

That all five variants of the first screen were ahead looks stronger than it is. The five leads share one baseline total, and that total, 19, is the lowest of the three baseline sessions on S (19, 21, 20). A low draw of the shared baseline lifts every lead at once.

This calculation is a normal approximation with a noise estimate from six runs per task, and it assumes that a variant has the same run-to-run noise as the baseline. We give it as a guide for planning, not as a test.

Consequences for the procedure. A screen of this size selects a candidate; it does not measure one. The leader's lead has to be discounted by the selection effect before any plan is built on it, and the first two confirmations show what can remain of it. To rank variants, the standard deviation of a difference would have to fall from 2.9 points to about one point, which needs roughly nine times the runs: about 830 runs per arm on tasks like those of S. That is larger than a confirmation block. We conclude that the screen should be kept small and cheap, as a filter for broken builds and a tie-break among variants, and that the number of confirmations, not the number of screens, limits what can be learned.

The third candidate. In its confirmation the third candidate solved 118 runs against 97 for the baseline. By task, it was ahead on 24 tasks, behind on 11 and level on 93. The difference of 21 runs is 2.9 standard deviations of the run-to-run noise of a difference on H (Section 8). Against the baseline session of the second confirmation, which solved 103 runs on the same tasks, the difference is 15 runs, or 2.0 standard deviations.

These ratios are post hoc, and they answer a narrower question than the rule does. Run-to-run noise asks whether the same tasks, run again, would give the same difference. The rule asks whether the gain would hold on other tasks and repositories, and its test, with repositories as clusters, gives $p = 0.0405$, above the registered level. The registered verdict stands: not confirmed. We do not call the third candidate an improvement. H has now been used for two confirmations, and the third candidate was built after the result of the first of them was known. A further test of this candidate needs tasks that no experiment has touched.

10.4 Recomputation from the released rows

The three confirmation blocks are in `runs.csv` with the rest. Appendix C recomputes every quantity of the three decisions from these rows with the released kit and compares it with the frozen reports; all 104 quantities agree. The p-values of the first two applications are equal to four decimals, 0.6034 and 0.6034. The recomputation confirms that this is a coincidence of two different data sets, with 63 against 62 solved runs of 288 in one and 103 against 101 of 512 in the other, and not a copying error.

11 Cross-experiment analysis

The experiments of Sections 7 to 10 share one task pool, one runner and, for E4B, one baseline. This section reads across them. It asks how large the noise of each kind of comparison is, which tasks can move at all, what a hosted score adds to a local result, and what the record implies for the test of a further candidate. Every analysis in this section was chosen after the results were known. None of them changes a registered verdict.

11.1 Three comparisons and their noise

Table 31 puts the three comparisons that we used side by side. All three hold the tasks fixed, so the numbers answer one question only: would the difference appear again if the same two builds ran again on the same tasks? For that question a confirmation block on the workstation is the most exact instrument that we have, and a pair of hosted scores is the least exact.

The pass rule asks a harder question: would the gain hold on tasks from repositories that the block did not contain? Its test resamples repositories, and its answer can differ widely from the answer for fixed tasks. The third application shows the size of that difference. The candidate solved 21 more runs than the baseline, a gain of +4.10 points. Against run-to-run noise alone this is 2.9 standard deviations. Under the repository-clustered test the one-sided p is 0.0405, above the limit of 0.025 (Section 10.3). Both statements are correct. The first says that the two builds very probably differ on the 128 tasks of H. The second says that the 19 repositories of H do not yet show that the gain carries over to other repositories.

A third source of error is selection. The candidate that enters a confirmation is the leader of a screen, and the leader of several equal arms is ahead by construction: 2.4 points on average when five variants are screened against the baseline, and 1.2 points when two are (Section 10.3). A reader who compares a screen lead with Table 31 must subtract this first.

11.2 Tasks that the small model does not solve

On most tasks the outcome of an E4B run does not depend on the run, and it does not depend on the arm. The baseline solved 83 of the 128 tasks of H in none of its eight runs. Each of these tasks was also run eight times by the two candidates that were tested on H. They solved 7 of them at least once, in 10 of 664 runs; 76 tasks stayed unsolved in all 16 runs. On the screen set S the baseline never solved 34 of 48 tasks in six runs, and 28 tasks stayed unsolved in the 24 runs of all arms of the three screens. Over all E4B runs on the workstation, 263 tasks were attempted, each in 6 to 31 runs. 107 of them were solved at least once. The other 59% were never solved.

"Never" is a statement about our runs. A task that is solved in one run of ten stays unsolved in eight runs with a probability of 43%. The few candidate runs that solved a task of the never-solved group are therefore what such tasks would give without any change to the agent, and we do not read them as tasks that a candidate opened.

Two consequences follow. First, the changes that we tested act on a minority of tasks. Even the largest gain, that of the third candidate, changed the solved count of 24 tasks upwards and of 11 tasks downwards, and left 93 of 128 as they were. Second, the task set fixes most of a solved share before any arm runs. The share of tasks that some arm solved in some run, 107 of 263, is about twice the baseline's solved share of runs (20.2%). It bounds what any choice among our variants could reach on these tasks. Section 8.6 showed how the failures look: in 93% of the unsolved workstation runs the agent handed in a patch that the tests reject, and in 0.4% it was stopped by the time limit.

11.3 What a hosted score adds

Precision. The census of repeated public draws gives a standard deviation of 1.56 tasks of 58 for one draw of unchanged 31B files (our recount of its 29 draws of six file sets). The difference of two submissions then has a standard deviation of 3.8 points. To detect a gain of two tasks of 58 with a power of 80% at a one-sided level of 0.025, each of the two files would need about 10 draws. At one submission per day this is twenty days for one comparison. The difference of two arms in one confirmation block has about 7 times less run-to-run variance than the difference of two single hosted draws, and the block takes three hours. For E4B files no repeated draws exist, so the first number is not known for the small model.

Relevance. The hosted score is measured on the tasks that decide the competition. Local runs are not. For the four arms with at least 70 local runs, the hosted score is between 0.22 and 0.34 of the local solved share (Table 11): 0.12 and 0.18 against 54% and 53% of the development tasks for the two 31B arms, and 0.06 and 0.05 against 20% and 23% of Berth runs for the two E4B arms. The hidden tasks need not resemble either local set. The development tasks are public, and the Berth tasks are those that load and pass validation. We did not test which of these reasons accounts for the gap.

Order. We have two pairs of arms with both a local and a hosted result. `W3` and `BF5` tied locally (38 and 37 of 70) and scored 0.12 and 0.18. The third E4B candidate was ahead of the baseline locally by +4.10 points and scored 0.05 against 0.06. In both pairs the hosted order is the reverse of the local order. The hosted difference of the 31B pair is about one and a half standard deviations of a difference of single draws (Section 7.7); that of the E4B pair is 0.01. Neither can be told from noise. Two pairs cannot show whether a local gain carries over to the hidden tasks. They show that one hosted draw per arm cannot answer the question.

11.4 Time limits and the two models

The time limit of a session shapes the results of the large model and hardly touches the small one. On the development tasks under emulated latency, 12.6% of the 31B runs were stopped by the limit; on Berth tasks on 4×L4, 8.7%. Of the E4B runs on the workstation, 0.3% were stopped by the time limit and 1.2% by the limit on tool calls (Table 17). For 31B the limit binds in about one run of eight, so the way an arm uses its time is part of its result. The first wrapped arm, for example, finished its jobs sooner than the reference because it wrote no history summaries (Section 7.1). For E4B, time is not the binding limit. The three candidates used 1.026, 1.182 and 1.103 times the agent time of the baseline, inside the factor of 1.2 that the pass rule allows. With so few runs stopped by a limit, a faster E4B agent could gain at most that fraction of runs. A change that helps must alter the patch that the agent submits.

11.5 What the record implies for a further candidate

1. **Confirmations are the scarce resource.** A screen costs an hour and ranks nothing (Section 10.3). A confirmation costs about a thousand runs and three hours, and it spends its panel: H has been used twice, and the third candidate was built after the first result on H was known.
2. **The validated pool is nearly used up.** Of the 579 validated tasks, 479 have been used by some experiment of this paper. The remaining 100 tasks lie in 6 repositories; one repository holds 68 of them and 3 repositories hold one task each. The pass rule asks for three repositories with a gain, so these tasks alone cannot carry a confirmation. A further 216 tasks were used only by the case study of Section 9, which compared prompt renderings and no candidate. They can serve as a fresh panel only for a candidate whose design did not draw on the per-task results of that study.
3. **The size of the panel sets the power.** For an even gain of three tasks of 58 the simulated pass rate of the rule is about 60% on the small panel and about 90% on the large one (Table 5). A gain that is concentrated in few repositories passes far less often.
4. **A hosted draw is a check of the file.** It shows that a bundle loads and runs under the scorer. It does not show which of two bundles is better (Section 11.3).

12 Related work

Benchmarks, agents and task pools. Repository-level coding benchmarks grade an agent by running tests in an environment built for each task. SWE-bench holds 2,294 tasks from 12 Python repositories [21], and a human-validated subset of 500 tasks is in wide use [31]. Agent frameworks and training environments bring their own sandboxes and task pools [32, 42, 44, 45], and SWE-rebench keeps collecting new tasks so that an evaluation can use tasks that a model has not seen [2]. For this competition, Berth [33] and FreshSWE-Py [30] supply tasks in the format of the harness, and a third data set reports on the soundness of tasks together with local results [14]. We built no task set. All tasks of our experiments come from the competition's development set and from Berth, and we add grading controls for every Berth task that we used (Table 13).

Evaluation practice. Audits of agent benchmarks show that validity has to be checked and cannot be assumed [47], that scores move with the evaluation setup and with the format of the prompt [3, 37], and that agent evaluations are often hard to reproduce [22]. We address one narrow case of this: whether a local stack reproduces one hosted scorer. We compare prompts as token

ids and not as message text, because equal strings need not give equal tokens [40] and because the control tokens of a chat template matter [20]. The checks run on the serving engine of the scorer [23]. The prompt-construction divergences that we measure in Section 9 are documented upstream [10, 13, 15, 35]. We did not discover them. We add executable checks for them and a measurement of their effect on E4B.

Isolation. MicroVMs [1] and user-space kernels [46] are alternatives to plain containers that are built for security, and the overhead of the latter has been measured [46]. Our runner uses plain containers; we tried neither alternative. Containers can consume resources outside their own accounting [11], which is the kind of route through shared host resources that our verifiers used (Section 4). SandboxEval [34] and SandboxEscapeBench [25] test sandboxes and models' ability to escape them.

Several paper-track resources overlap this one (described from their public text only): LAB [26], a Docker sandbox with 11 tests and a bundle gate with 16 detectors validated by fault injection; Szeto et al.'s cloud rig with 141 run folders [38]; gemma4-swe-kit [6], a proxy reproducing the scorer's template, call parsing and compaction on other servers; a dataset card reporting count parity on 107 captured vLLM requests [41]; Silent Zeros [19], a bundle checker tested on 22 broken bundles; and ThoughtKeep [18], 1,470 Gemma 4 31B attempts on thought retention. We did not find in those texts, and offer here: a runner attacked by independent verifiers with its canary misses published, token-id comparison backed by a mutation suite, and a trap measurement on E4B with a per-job prompt check.

Statistics of comparisons. We follow the advice to treat an evaluation as an experiment, with clustered and paired errors and a power analysis [28], and to size decisions to measured noise, as in signal-to-noise analyses of benchmarks [17]. The test of the pass rule is a wild cluster bootstrap-t over repositories [4] with Rademacher weights [8]. Few or unbalanced clusters can break the nominal size of such tests [5, 24], and with few clusters only a small set of p-values can be attained [43]; for that reason the rule's size and power were simulated on the frozen panels (Table 5). The estimates of leaderboard noise for this competition come from third parties [7, 29]. What this version adds to them is the local counterpart: the run-to-run noise of one fixed agent on fixed tasks, measured by repeated sessions (Section 11.1), and three worked cases of a screen lead that was tested on new tasks (Section 10.3).

13 Limitations

The limitations of the kit and of the case study are those of the short paper.

- **Solve-level parity is not refuted, not established.** The evidence (Figure 10A) is one 24-task list, one repeat per run; ten runs solved only six distinct tasks.
- **Kaggle-side facts are partly inferred**: the hosted google-adk release, tool-schema equality, and the identity of the hosted template and weights with the public Kaggle model.
- **Environment differences remain.** One RTX 5090 instead of four L4s; 2 CPUs and 10 GiB per job; a bare `pytest` command, used in 30 of 216 commands in our Kaggle traces, is absent locally.
- **The case study** covers one model, one bundle, one google-adk release and one configuration per divergence. Arm T swaps a whole template; in arm H thoughts still reach compaction summaries.

The record of experiments adds the following.

- **Rows that are missing.** One 31B block, an arm with a ten-minute limit that solved 18 of 30 development tasks, has no result rows that we could retrieve. It is in no table and in no released file. The rows of one workstation parity run under runner version 2 (4 of 24 solved) were not kept either; that run appears only in Figure 10.
- **One run per task for 31B.** On the development tasks every 31B configuration ran each task once. Differences between configurations and differences between runs cannot be separated there (Section 7).
- **Arms and sessions are confounded in places.** Some 31B arms ran in blocks of their own, under a different load. The E4B candidates always ran in one job queue with the baseline.
- **Task sets were not always chosen by a recorded rule.** The tasks of the two 31B blocks on real hardware were picked with earlier results in view. For the eight tasks of block `berthE4-k2` the method of selection was not recorded. The screen set S is not fresh: 32 of its 48 tasks had been run in earlier blocks. The development block of the third programme used 16 tasks of the small panel after that panel's confirmation.
- **The hold-out H was used twice.** The third candidate was built after the result of the first confirmation on H was known. Its confirmation was registered with this disclosure, and its verdict is "not confirmed".
- **Post hoc analyses.** The noise estimates, the selection effect of Section 10.3 and all of Section 11 were chosen after the results were known. They describe the record. They are not tests.
- **One baseline.** The share of tasks with an uncertain outcome, and with it the run-to-run noise, was measured for one E4B agent on Berth tasks. Another agent, another model or another task source can have a different share.
- **Hosted scores are single draws.** We have one hosted draw per file and no repeated draw of an E4B file.
- **Machines differ between experiments.** The 31B runs on development tasks used emulated latency on a single GPU, the first E4B blocks used hosted notebooks with four L4 GPUs, and most E4B runs used the workstation. Times are comparable within a block and not across machines.
- **Arms cannot be rebuilt from this paper.** Prompts, the E4B bundle and the adapter are withheld (Section 2), and each arm is described by the kind of its change. The rows of the development tasks carry opaque task codes, because the competition data may not be redistributed; they support counts and pairings and cannot be joined with the task texts.

14 Availability and released data

`drydock-eval` 0.4.0 (Apache-2.0) has three modules: `conform` (checks, mock server, mutation suite), `contain` (a reference runner tree, its escape tests, a gate helper) and `compare` (pass rule, sign-flip tests, simulations). Its 435 CPU tests pass. The released runner has not been run in a container as released: the 200 of 200 result is for the deployed tree, of which the release is an identifier-substituted copy with two canaries rewritten. Code: `https://github.com/MinhThanghus/drydock-eval` (tag `v0.4.0`; branch `paper`: supplement, Section 9 outcome rows). The package also carries copies of the Section 9 protocol, analysis code

and job-list generator, redacted where needed, each with its original's sha256; the agent bundle and request dumps are planned for 2026-12-03.

Data of this version. The branch `paper` of the repository also holds this extended version with its sources, its figures and the data behind every table and figure (Appendix D): `runs.csv` with one row for each of the 5,024 agent runs of Sections 7, 8 and 10, `controls.csv` with the 2,494 grading controls, `serving_bench.csv` with the 72 serving measurements, the tables of blocks, arms and hosted submissions, the three applications of the pass rule, and the 4,636 outcome rows of the case study. A row of `runs.csv` gives the block, the arm, the task and its repository, the outcome, the way the session ended, times, token counts and the number of tool calls. It holds no prompt, no model output, no patch and no error text.

Scripts. `scripts/analyze_long.py` computes every number and every table of Sections 3, 7, 8, 10 and 11 from these files and stops if a check against the rows fails. `scripts/recheck_gates.py` applies the pass rule of the released kit to the rows of the three confirmation blocks and compares the result with the registered decisions (Appendix C). `figsrc/figs_long.py` draws the figures. The passages taken from the short paper are copied by `scripts/reuse_short.py` and are not edited by hand.

Withheld. Not released are the prompts, the agent bundles and their changed versions, the adapter and its training data, the request dumps, and the table that maps the task codes of the development set to the tasks of the competition.

Licences. Code: Apache-2.0. Text, figures and data: CC BY 4.0.

15 Conclusion

`drydock-eval` gives a team with one GPU three things: a runner that has been attacked and whose misses are published, checks that compare a local serving stack with the scorer down to token ids, and a pass rule with known error rates. This extended version adds what the kit was used for: 5,024 agent runs in 31 blocks and 37 arms, and the 4,636 rows of the case study.

Most of what the record shows is the absence of a difference. Six configurations of a 31B agent lay between 37 and 44 of 70 development tasks, and no pairwise difference was detected. Four prompt divergences changed the E4B solve rate by no detectable amount. Three changed E4B agents led their screens and were not confirmed on new tasks, with gains of -0.35, -0.39 and $+4.10$ points. The third of them is the closest case: its gain is large against run-to-run noise and does not pass the test across repositories. We do not call it an improvement.

The record also explains why this is the expected outcome at this scale. On a fixed set of tasks the small model is nearly deterministic at the level of a task: about two thirds of the tasks were solved in no run, and the baseline repeated its total within 1.17 points. A screen of 96 runs per arm cannot rank variants, and its leader is ahead by selection. A hosted score is one draw with a standard deviation near 1.56 tasks of 58. The instrument that can decide is a confirmation on tasks from repositories that no earlier experiment has used, and each one spends its tasks. The validated pool that we used is almost spent.

We release every run-level result so that others can recompute these numbers, plan the size of their own comparisons from the measured noise, and avoid paying again for the same screens.

References

[1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, 2020. URL https://www.usenix.org/conference/nsdi20/presentation/agache. Accessed 2026-11-09.

[2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swerebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 26420–26466, 2025. doi: 10.52202/085713-0788. URL https://arxiv.org/abs/2505.20411. Accessed 2026-11-09.

[3] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, Charles Lovering, Niklas Muennighoff, Ellie Pavlick, Jason Phang, Aviya Skowron, Samson Tan, Xiangru Tang, Kevin A. Wang, Genta Indra Winata, François Yvon, and Andy Zou. Lessons from the trenches on reproducible evaluation of language models. arXiv preprint, 2024. URL https://arxiv.org/abs/2405.14782. Accessed 2026-11-09.

[4] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008. doi: 10.1162/rest.90.3.414. URL https://doi.org/10.1162/rest.90.3.414. Accessed 2026-11-09.

[5] Ivan A. Canay, Andres Santos, and Azeem M. Shaikh. The wild bootstrap with a "small" number of "large" clusters. *The Review of Economics and Statistics*, 103(2):346–363, 2021. doi: 10.1162/rest_a_00887. URL https://doi.org/10.1162/rest_a_00887. Accessed 2026-11-09.

[6] damsolanke (GitHub user). gemma4-swe-kit. GitHub repository (Apache-2.0), 2026. URL https://github.com/damsolanke/gemma4-swe-kit. Accessed 2026-11-09.

[7] dariushafshar (Kaggle user). Gemma 4 agent: Public config draws. Kaggle dataset (CC BY 4.0) and notebook 'Gemma 4: Most-Run Public Config Averages 10/58', 2026. URL https://www.kaggle.com/datasets/dariushafshar/gemma-4-agent-public-config-draws. Accessed 2026-11-09.

[8] Russell Davidson and Emmanuel Flachaire. The wild bootstrap, tamed at last. *Journal of Econometrics*, 146(1):162–169, 2008. doi: 10.1016/j.jeconom.2008.08.003. URL https://doi.org/10.1016/j.jeconom.2008.08.003. Accessed 2026-11-09.

[9] delta9000 (GitHub user). vLLM issue #38488, 2026. URL https://github.com/vllm-project/vllm/issues/38488.

[10] dimondev. fix: chat_template, keep tool-call reasoning across later user turns. Hugging Face pull request #44 on google/gemma-4-E2B-it, opened 2026-07-28 (open), 2026. URL https://huggingface.co/google/gemma-4-E2B-it/discussions/44. Accessed 2026-11-09.

[11] Xing Gao, Zhongshu Gu, Zhengfa Li, Hani Jamjoom, and Cong Wang. Houdini's escape: Breaking the resource rein of linux control groups. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*, pages 1073–1086, 2019. doi: 10.1145/3319535.3354227. URL https://doi.org/10.1145/3319535.3354227. Accessed 2026-11-09.

[12] Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026.

[13] Georgi Gerganov. Chat template issue: extra newline character when rendering thinking? Hugging Face discussion #135 on google/gemma-4-31B-it, opened 2026-07-16, 2026. URL https://huggingface.co/google/gemma-4-31B-it/discussions/135. Accessed 2026-11-09.

[14] Dmitrii Gluzdov. Gemma 4 task soundness and local results. Kaggle dataset dmitriigluzdov/gemma-4-task-soundness-and-local-results (CC BY 4.0), 2026. URL https://www.kaggle.com/datasets/dmitriigluzdov/gemma-4-task-soundness-and-local-results. Accessed 2026-11-09.

[15] Google AI for Developers. Gemma 4 prompt formatting. Gemma documentation, accessed 2026-11-09, 2026. URL https://ai.google.dev/gemma/docs/core/prompt-formatting-gemma4. Accessed 2026-11-09.

[16] Google DeepMind. Google - the gemma 4 developer agent competition. Kaggle competition, 2026. URL https://www.kaggle.com/competitions/gemma-4-developer-agent. Accessed 2026-11-09.

[17] David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah A. Smith, Hannaneh Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and noise: A framework for reducing uncertainty in language model evaluation. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 19286–19327, 2025. doi: 10.52202/085713-0576. URL https://arxiv.org/abs/2508.13144. Accessed 2026-11-09.

[18] hemantkumarbk (Kaggle user). Coding-agent thought retention: Reproduction data. Kaggle dataset, 2026. URL https://www.kaggle.com/datasets/hemantkumarbk/thoughtkeep-reproduction-inputs.

[19] houchiunyo (Kaggle user). Gemma 4 agent resource: runs & checker. Kaggle dataset, 2026. URL https://www.kaggle.com/datasets/houchiunyo/gemma4-agent-silent-zeros.

[20] Hugging Face. Chat templates (transformers documentation). Online documentation, accessed 2026-11-09, 2026. URL https://huggingface.co/docs/transformers/main/en/chat_templating. Accessed 2026-11-09.

[21] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.06770. Accessed 2026-11-09.

[22] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. Ai agents that matter. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.01502. Accessed 2026-11-09.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165. Accessed 2026-11-09.

[24] James G. MacKinnon and Matthew D. Webb. Wild bootstrap inference for wildly different cluster sizes. *Journal of Applied Econometrics*, 32(2):233–254, 2017. doi: 10.1002/jae.2508. URL https://doi.org/10.1002/jae.2508. Accessed 2026-11-09.

[25] Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. Quantifying frontier llm capabilities for container sandbox escape. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), PMLR 306*, pages 86682–86713, 2026. URL https://proceedings.mlr.press/v306/marchand26a.html. Accessed 2026-11-09.

[26] marcofpereira (Kaggle user). Gemma 4 developer agent lab - instrumentos. Kaggle dataset marcofpereira/gemma4-lab-v1 (paper-track companion; MIT per its text), 2026. URL https://www.kaggle.com/datasets/marcofpereira/gemma4-lab-v1. Accessed 2026-11-09.

[27] Elan Markowitz, Bryan Perozzi, Benedek Rózemberczki, Glenn Cameron, Hadi Hemmati, Yuchen Li, Michael Galkin, Majid Farhadi, Ryan Holbrook, and Ashley Oldacre. Google - the gemma 4 developer agent paper track. Kaggle competition, 2026. URL https://www.kaggle.com/competitions/gemma-4-developer-agent-paper. Accessed 2026-11-09.

[28] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv preprint, 2024. URL https://arxiv.org/abs/2411.00640. Accessed 2026-11-09.

[29] Mobassir. Too small to see: Resolution limits, leaky statements and re-sent reasoning when tuning a gemma 4 31b software-engineering agent. Gemma 4 Developer Agent Paper Track writeup and companion dataset 'SWE Agent Eval Validity: Gemma 4 Tasks & Results' (Apache-2.0); author line on the card: Syed Mobassir Hossain, Hiperdyne Corporation; the author's own BibTeX uses author = Mobassir, 2026. URL https://www.kaggle.com/datasets/mobassir/swe-agent-eval-validity-gemma4. Accessed 2026-11-09.

[30] moshegolderson (Kaggle user). Freshswe-py: Scorer-faithful tasks and three ways a coding leaderboard misleads. Gemma 4 Developer Agent Paper Track writeup; dataset 'FreshSWE-Py: post-cutoff SWE tasks for Gemma 4', 2026. URL https://www.kaggle.com/datasets/moshegolderson/freshswe-py. Accessed 2026-11-09.

[31] OpenAI. Introducing swe-bench verified. OpenAI blog post (2024-08-13 per search index), 2024. URL https://openai.com/index/introducing-swe-bench-verified/. Accessed 2026-11-09.

[32] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *International Conference on Machine Learning (ICML 2025)*, 2025. URL https://arxiv.org/abs/2412.21139. Accessed 2026-11-09.

[33] Jay Patel. Berth: Harness-compatible, post-cutoff python tasks for offline gemma 4 coding agents. Gemma 4 Developer Agent Paper Track, Kaggle; dataset jayp2005/berth-swe-tasks

(Apache-2.0), 2026. URL https://www.kaggle.com/datasets/jayp2005/berth-swe-tasks. Accessed 2026-11-09.

[34] Rafiqul Rabin, Jesse Hostetler, Sean McGregor, Brett Weir, and Nick Judd. Sandboxeval: Towards securing test environment for untrusted code. arXiv preprint (preliminary version, working paper), 2025. URL https://arxiv.org/abs/2504.00018. Accessed 2026-11-09.

[35] rladeira. [litellm] role='tool_responses' for gemma 4 breaks tool calling on vertex ai model garden maas (400 invalid_argument) (issue #7977) and fix(models): use standard tool role for gemma 4 on vertex ai (pull request #7978). google/adk-python GitHub issue #7977 (opened 2026-11-03, closed 2026-11-05) and pull request #7978 (merged via Copybara as commit b33b43e, 2026-11-05), 2026. URL https://github.com/google/adk-python/issues/7977. Accessed 2026-11-09.

[36] ryanholbrook (Kaggle user). Getting started - Gemma 4 developer agent. Kaggle notebook, 2026. URL https://www.kaggle.com/code/ryanholbrook/getting-started-gemma-4-developer-agent.

[37] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.11324. Accessed 2026-11-09.

[38] Jayden Szeto, Daniel Guo, and Hayley Szeto. Scorer-faithful local evaluation for small coding agents. Gemma 4 Developer Agent Paper Track writeup; dataset 'Gemma 4 agent: scorer-faithful rig and run logs' (Apache-2.0), 2026. URL https://www.kaggle.com/datasets/jaydenszeto123/gemma-4-developer-agent-scorer-faithful-rig. Accessed 2026-11-09.

[39] Team Muhammed Idris. gemma4-agent-runs. Kaggle dataset (Apache-2.0), 2026. URL https://www.kaggle.com/datasets/muhammedidris28/gemma4-agent-runs.

[40] The Agent Lightning (AGL) Team. No more retokenization drift: Returning token ids via the openai compatible api matters in agent rl. vLLM Blog, 2025-10-22, 2025. URL https://vllm.ai/blog/2025-10-22-agent-lightning. Accessed 2026-11-09.

[41] tyfeng1997 (Hugging Face user). gemma4-e4b-swe-sft-32k-competition-template. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/tyfeng1997/gemma4-e4b-swe-sft-32k-competition-template.

[42] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR 2025)*, 2025. URL https://arxiv.org/abs/2407.16741. Accessed 2026-11-09.

[43] Matthew D. Webb. Reworking wild bootstrap-based inference for clustered errors. *Canadian Journal of Economics/Revue canadienne d'économique*, 56(3):839–858, 2023. doi: 10.1111/caje.12661. URL https://doi.org/10.1111/caje.12661. Accessed 2026-11-09.

[44] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024. doi: 10.52202/079017-1601. URL https://arxiv.org/abs/2405.15793. Accessed 2026-11-09.

[45] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 107402–107452, 2025. doi: 10.52202/085713-3239. URL https://arxiv.org/abs/2504.21798. Accessed 2026-11-09.

[46] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The true cost of containing: A gvisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, 2019. URL https://www.usenix.org/conference/hotcloud19/presentation/young. Accessed 2026-11-09.

[47] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 184435–184475, 2025. doi: 10.52202/085713-5547. URL https://arxiv.org/abs/2507.02825. Accessed 2026-11-09.

A Arms and blocks

Table 32 lists every arm with the number of its runs and what was changed against the arm it was built from. Table 33 lists every block. The same two tables are in the released files `arms.csv` and `experiments.csv`.

B Recomputation of the case study with the released kit

The released table `e8_compare_table.csv` is in the input format of the `compare` module of the kit. From a clone of the repository, two commands recompute the analysis:

```
pip install "./drydock-eval[compare]"
drydock-eval compare signflip e8_compare_table.csv --reference F --repeats 4
```

We ran this command from the released source tree with the script `supp/reproduce.py`, which stops on any mismatch. For all four arms the command returned the registered estimate, the exact p as the same fraction, the Holm-adjusted p and every repository-level quantity of Table 24.

Two quantities depend on Monte Carlo draws: the inverted intervals, and the Monte Carlo p that is reported next to the exact one. The draws are frozen (seed 20261109), but which draw meets which task depends on the order of the tasks. The registered analysis used the order of the panel file; the command sorts the task identifiers. With the tasks in panel order, the routine of the kit returns the registered values to the last digit (Table 34). With the order of the command,

the ends of the 95% intervals move by at most 0.13 points and those of the simultaneous intervals by at most 0.02 points. The largest move is the upper end of the interval of H, +0.25 instead of +0.38 points. The p-value of the test is a step function of the shift, and across that stretch it stays between 0.0501 and 0.0509, next to the 0.05 it is compared with. Both intervals include zero. The paper reports the registered intervals.

C The three decisions recomputed from the released rows

The frozen reports of the three applications of the pass rule were written by the registered analysis scripts on the machine that ran the jobs, from result files that are not released. The released file `runs.csv` holds the same runs with a reduced set of columns. The script `scripts/recheck_gates.py` builds, for each application, the input table of the `compare` module from these rows, calls the routines of the kit at tag `v0.4.0` with the registered thresholds and seed, and compares the result with the released copy of the frozen report, `pass_rule_applications.json`. It stops on the first mismatch.

All 104 compared quantities agree: for each application the solved counts of both arms, the gain, the bootstrap p with its 9,999 draws, the gain and the leave-one-out estimate of every repository, the counts of repositories that gain and lose, and the decision. Table 35 gives the values. The time ratio is the one quantity that the recomputation can only approximate, because the reports computed it from timings with more digits than the released rows keep; recomputed from the released rows it is 1.026, 1.182 and 1.103, which round to the reported values.

D Released data

Table 36 lists the files. `data/README.md` describes every column, and `data/SHA256SUMS` holds the checksums. The file `runs.csv` has 29 columns:

`experiment, block` the experiment family and the block (Table 33).

`model, machine, task_source` 31B or E4B; the machine of Section 2; development tasks or Berth.

`arm, condition` the arm (Table 32); `standard`, or `masked_test_runners` for the runs of Table 8.

`task, repository, repeat` the task (an opaque code for development tasks, the Berth identifier otherwise), its repository, and the number of the repeat within the block.

`resolved` 1 if the tests of the task passed on the patch that the run left behind, else 0.

`end_reason, outcome_id` how the run ended (normally, at the time limit, at the tool-call limit, without a submission), and the outcome class of the runner.

`wall_s, t_agent_s, t_setup_s, t_verify_s` seconds of the whole job and of its three phases, where the harness recorded them.

`emulation_pad_s` seconds of padding added to model calls on the emulated-latency machine.

`model_calls, model_calls_failed, tool_calls` counts per run.

`tokens_in, tokens_out, max_prompt_tokens` prompt tokens summed over the requests of the run, generated tokens, and the longest prompt.

`summaries` number of history summaries that the harness made in the run.

`patch_chars, test_exit` length of the final patch, and the exit status of the test command.

`start_utc, end_utc` times of the job, where recorded.

No column holds prompt text, model output, error text, a path or a machine name. The columns of `e8_runs.csv` are listed in Table 37.

Table 16: Solved share of the E4B baseline by repository, over every baseline run of the screen and confirmation blocks on the workstation. The interval treats runs as independent and is too narrow where few tasks carry many runs.

Repository	Tasks	Runs	Solved	%	95% interval
executablebooks/markdown-it-py	6	36	19	52.8	37–68
Kludex/starlette	1	6	3	50.0	19–81
jd/tenacity	15	98	33	33.7	25–43
mahmoud/boltons	17	118	38	32.2	24–41
pypa/packaging	24	128	36	28.1	21–36
marshmallow-code/marshmallow	8	56	15	26.8	17–40
pytest-dev/pluggy	12	80	21	26.2	18–37
python-humanize/humanize	12	78	20	25.6	17–36
python-attrs/attrs	6	40	9	22.5	12–38
python-poetry/tomlkit	17	106	22	20.8	14–29
jazzband/prettytable	12	70	13	18.6	11–29
pydantic/pydantic-settings	24	156	27	17.3	12–24
fastapi/typer	5	36	6	16.7	8–32
more-itertools/more-itertools	20	124	20	16.1	11–24
python-jsonschema/jsonschema	1	8	1	12.5	2–47
python-attrs/cattrs	11	80	9	11.2	6–20
pallets/click	23	142	14	9.9	6–16
pallets/werkzeug	17	126	12	9.5	6–16
hynek/structlog	10	70	5	7.1	3–16
arrow-py/arrow	3	22	0	0.0	0–15
pyparsing/pyparsing	3	14	0	0.0	0–22
tkem/cachetools	1	6	0	0.0	0–39

Table 17: Time, tokens and endings of the agent runs by model and setting. Time: seconds per run (whole job for the development tasks, agent phase for Berth). In: prompt tokens summed over the requests of a run, in thousands. T/O, Budget, Empty: percentage of runs stopped by the time limit, stopped by the tool-call limit, and left without a patch.

Setting	Runs	Solved %	Mean	Median	Solved	Unsolved	Out	In (k)	Tools	T/O	Budget	Empty
31B, development tasks, emulated latency	570	57.2	278	255	231	341	7,089	240	20.6	12.6	0.4	13.5
31B, development tasks, 4×L4	36	58.3	286	256	234	358	8,515	264	23.8	11.1	2.8	11.1
31B, Berth, 4×L4	104	38.5	226	212	165	265	7,495	196	19.2	8.7	0.0	4.8
E4B, development tasks, 4×L4	72	20.8	117	106	85	125	4,665	102	9.8	1.4	0.0	4.2
E4B, Berth, 4×L4	160	20.0	73	61	43	80	6,261	110	10.6	0.0	1.9	11.9
E4B, Berth, workstation	4,082	21.3	75	60	45	84	8,146	156	14.2	0.3	1.2	5.5

Table 18: Arms; changes in percentage points.

Arm	Difference from the scorer path	Requests changed by the divergence	Change against F [95% interval]; Holm p
F	none: role `tool`, Kaggle template, compaction at 14,336 tokens, thoughts under both keys	—	solve rate 23.8%
R	model name contains "gemma4": tool results get role `tool_responses`	89%	−0.3 [−2.8, +2.2]; p = 1
C	compaction unset: prompts grow until rejected above 24,576 tokens (the run ends; edits are still graded)	21%	+0.2 [−2.0, +2.5]; p = 1
H	history thoughts removed from every request, as when a client omits the key the server reads	90%	−2.1 [−4.4, +0.4]; p = 0.44
T	the Hugging Face E4B template: a thought opener is forced after each tool result	85%	+0.8 [−1.4, +2.9]; p = 1

Table 19: Execution record: the block ran as four batches and one batch of re-runs. An interrupted job is an infrastructure row, with or without a written result row; its slot was re-run once.

Batch	Jobs started	Analysed runs	Interrupted, row written	Interrupted, no row	How the batch ended
1	967	959	0	8	runner reached its open-file limit
2	378	370	8	0	external termination signal
3	3,074	3,066	8	0	model server ended (out of memory)
4	201	201	0	0	job list completed
5	24	24	0	0	re-runs of the interrupted jobs completed

Table 20: Outcome of every analysed run, by arm (one row per task, arm and repeat).

Arm	Solved	Unsolved, run completed	Five-minute limit	Over-long prompt rejected	Tool-call budget (28)	No submission	Runs
F	220	676	17	0	6	5	924
R	217	677	20	0	9	1	924
C	222	568	17	114	1	2	924
H	201	597	106	1	18	1	924
T	227	668	19	0	4	6	924

Only the runs of the first column are solved; the other columns split the unsolved runs by how the run ended. A run that ended at a limit was still graded on the edits it had made: 5 solved runs lasted the full five minutes (all in arm H), and 86 of the 179 runs unsolved at that limit had left edits that were tested.

Table 21: Solve rate per arm: task-weighted mean with a 95% interval from a repository-clustered (CR1) standard error; solved runs per repeat.

Arm	Solve rate (%)	95% interval	Solved runs	Repeat 1	2	3	4	Runs
F	23.81	[16.7, 30.9]	220	50	59	57	54	924
R	23.48	[17.7, 29.3]	217	53	53	54	57	924
C	24.03	[16.6, 31.5]	222	58	56	55	53	924
H	21.75	[15.9, 27.6]	201	50	49	54	48	924
T	24.57	[18.6, 30.5]	227	59	56	51	61	924

Table 22: Tasks by the number of solved repeats (of four), per arm.

Arm	0 solved	1 solved	2 solved	3 solved	4 solved	Tasks
F	149	16	21	18	27	231
R	152	14	21	15	29	231
C	151	15	18	17	30	231
H	155	21	13	14	28	231
T	151	14	13	25	28	231

135 of the 231 tasks were solved in no run of any arm (0 of 20 runs) and 13 in every run; the other 83 tasks carry every difference between arms.

Table 23: Primary analysis: change in solve rate against F in percentage points, tasks with fewer and with more solved repeats than in F, exact two-sided task-level sign-flip test, Holm adjustment over the four arms, and intervals from inverting the test.

Arm	d	Exact d	Tasks lower; higher	Exact p	Holm p	Monte Carlo p	95% interval	98.75% simultaneous
R	−0.32	−1/308	26; 33	0.8652	1.000	0.8650	[−2.82, +2.17]	[−3.51, +2.85]
C	+0.22	1/462	26; 22	0.9253	1.000	0.9243	[−2.03, +2.48]	[−2.65, +3.09]
H	−2.06	−19/924	34; 20	0.1102	0.441	0.1106	[−4.45, +0.38]	[−5.09, +0.95]
T	+0.76	1/132	25; 30	0.5604	1.000	0.5599	[−1.42, +2.94]	[−2.02, +3.54]

Table 24: Secondary analysis at the repository level (15 repositories): exact wild cluster bootstrap-t with Rademacher weights, its inverted 95% interval, the six-point (Webb) variant, a CR1 interval, the repository-equal estimate and the range of leave-one-repository-out estimates. Estimates in percentage points.

Arm	Repositories that differ	Exact p	Webb p	95% interval	CR1 interval	Repository-equal	Leave-one-out range
R	12	0.8188	0.7780	[−3.81, +2.38]	[−2.70, +2.05]	+1.62	[−1.09, +0.36]
C	8	0.8438	0.7788	[−0.88, +2.81]	[−1.22, +1.65]	+4.02	[−0.22, +0.60]
H	12	0.1831	0.1736	[−4.94, +1.56]	[−4.65, +0.53]	−2.98	[−3.01, −1.44]
T	9	0.8047	0.7374	[−1.44, +4.62]	[−2.09, +3.61]	+2.65	[−0.55, +1.66]

Table 25: The 15 repositories of the panel: tasks, runs solved in arm F (of four per task) and, per arm, the summed change against F in task-equivalents (solved runs divided by four).

Repository	Tasks	F solved runs	R	C	H	T
`pypa/packaging`	65	65 of 260	+0.50	−0.50	−2.00	−1.00
`pydantic/pydantic-settings`	48	27 of 192	+1.25	0	+0.75	+2.75
`python-poetry/tomlkit`	35	31 of 140	−1.00	−0.50	−1.50	0
`more-itertools/more-itertools`	23	39 of 92	−1.50	+0.25	−1.75	0
`pallets/click`	22	13 of 88	−0.50	−0.25	−0.50	−0.25
`pallets/werkzeug`	9	3 of 36	+0.25	0	−0.25	+0.25
`mahmoud/boltons`	7	12 of 28	−0.25	0	+0.50	−0.25
`jd/tenacity`	6	9 of 24	−0.25	0	−0.25	−0.50
`pytest-dev/pluggy`	6	3 of 24	+0.50	+1.00	+0.75	0
`python-humanize/humanize`	3	11 of 12	−0.25	+0.25	+0.25	+0.25
`hynek/structlog`	2	2 of 8	+0.25	−0.25	−0.50	+0.25
`python-attrs/attrs`	2	0 of 8	0	0	0	0
`jazzband/prettytable`	1	1 of 4	+0.25	+0.50	−0.25	+0.25
`marshmallow-code/marshmallow`	1	0 of 4	0	0	0	0
`python-attrs/cattrs`	1	4 of 4	0	0	0	0
All	231	220 of 924	−0.75	+0.50	−4.75	+1.75

Table 26: Pre-registered sensitivity analyses of the primary estimate (percentage points; exact two-sided sign-flip p, unadjusted).

Arm	Analysis	Tasks	Runs affected	d	Exact p
R	five-minute-limit runs of both arms treated as missing	229	37	−0.29	0.8652
	slots of the arm with an infrastructure row scored unsolved	231	5	−0.43	0.8011
	slots of F with an infrastructure row scored unsolved	231	3	−0.22	0.9321
C	five-minute-limit runs of both arms treated as missing	229	34	+0.22	0.9253
	slots of the arm with an infrastructure row scored unsolved	231	5	+0.11	1.0000
	slots of F with an infrastructure row scored unsolved	231	3	+0.32	0.8506
H	five-minute-limit runs of both arms treated as missing	226	123	−1.25	0.3414
	slots of the arm with an infrastructure row scored unsolved	231	8	−2.16	0.0980
	slots of F with an infrastructure row scored unsolved	231	3	−1.95	0.1270
T	five-minute-limit runs of both arms treated as missing	229	36	+0.76	0.5604
	slots of the arm with an infrastructure row scored unsolved	231	3	+0.76	0.5604
	slots of F with an infrastructure row scored unsolved	231	3	+0.87	0.4947

Table 27: Exposure: requests re-rendered by the frozen checker, and those whose rendered prompt the arm's divergence changed.

Arm	Runs	Requests checked	Changed by the divergence	Share (%)	Failed calls recorded
F	924	15,043	0	0.0	17
R	924	15,290	13,616	89.1	20
C	924	13,036	2,709	20.8	131
H	924	17,450	15,684	89.9	112
T	924	14,787	12,629	85.4	19

No run has more than one failed call. Of the 299 failed calls, 178 are in runs unsolved at the five-minute limit, 115 in runs that ended on a rejected over-long prompt and 6 in solved runs.

Table 28: Exploratory run measures: mean in arm F, and the paired difference of each arm against F; below it, its 95% interval (CR1, repository clusters). No multiplicity adjustment.

Measure	F mean	R − F	C − F	H − F	T − F
history summaries per run	0.69	+0.13	−0.69	−0.30	−0.02
		[+0.04, +0.22]	[−0.86, −0.53]	[−0.47, −0.12]	[−0.10, +0.06]
runs with a summary (share)	0.482	+0.038	−0.482	−0.161	−0.014
		[−0.024, +0.099]	[−0.572, −0.391]	[−0.249, −0.073]	[−0.059, +0.030]
largest prompt (tokens)	12,721	+220	+2,160	−1,499	−95
		[+111, +329]	[+1,666, +2,655]	[−1,903, −1,095]	[−284, +95]
over-long prompt rejected (share)	0.000	+0.000	+0.123	+0.001	+0.000
		[+0.000, +0.000]	[+0.075, +0.172]	[−0.001, +0.003]	[+0.000, +0.000]
five-minute limit (share)	0.018	+0.003	+0.000	+0.096	+0.002
		[−0.005, +0.012]	[−0.005, +0.005]	[+0.073, +0.119]	[−0.002, +0.006]
tool-call budget reached (share)	0.006	+0.003	−0.005	+0.013	−0.002
		[−0.004, +0.011]	[−0.012, +0.001]	[−0.002, +0.028]	[−0.006, +0.002]
no submission (share)	0.005	−0.004	−0.003	−0.004	+0.001
		[−0.015, +0.006]	[−0.013, +0.007]	[−0.015, +0.006]	[−0.010, +0.012]
agent time (s)	77.8	+3.9	−11.2	+58.9	−1.2
		[−0.1, +7.9]	[−14.7, −7.8]	[+49.4, +68.3]	[−6.5, +4.0]
model calls	16.28	+0.27	−2.17	+2.60	−0.28
		[−0.25, +0.79]	[−2.51, −1.83]	[+1.31, +3.90]	[−1.05, +0.50]
failed model calls	0.018	+0.003	+0.123	+0.103	+0.002
		[−0.005, +0.012]	[+0.077, +0.169]	[+0.079, +0.127]	[−0.002, +0.006]
tool calls	12.87	+0.19	−1.23	+2.52	−0.23
		[−0.31, +0.70]	[−1.42, −1.05]	[+1.64, +3.41]	[−0.92, +0.45]
prompt tokens, summed	136,123	+3,585	+9,332	+12,785	−3,332
		[−1,493, +8,662]	[+4,824, +13,841]	[+443, +25,127]	[−11,389, +4,725]
completion tokens	7,690	+440	−1,253	+6,707	−245
		[−18, +899]	[−1,635, −871]	[+5,645, +7,768]	[−796, +305]

Table 29: Pilot, development and screen blocks of the E4B candidates on the workstation. Lead: difference to the baseline of the same block in percentage points. Agent: mean seconds of the agent phase.

Block	Arm	Tasks	Runs	Solved	%	Lead	Agent
pilot-2a	v-t1	15	15	4	26.7	—	98
pilot-2a	v-t2	15	15	5	33.3	—	71
pilot-2b	v-t3	15	15	3	20.0	—	96
pilot-2b	v-t4	15	15	7	46.7	—	89
pilot-2b	v-t5	15	15	6	40.0	—	77
pilot-2b	v-t6	15	15	6	40.0	—	66
dev-3	base	16	16	4	25.0	—	58
dev-3	v-a1	16	16	2	12.5	−12.5	48
dev-3	v-a2	16	16	3	18.8	−6.2	54
screen-1	base	48	96	19	19.8	—	64
screen-1	v-b1	48	96	22	22.9	+3.1	72
screen-1	v-p1	48	96	24	25.0	+5.2	64
screen-1	v-p2	48	96	25	26.0	+6.2	75
screen-1	v-p3	48	96	22	22.9	+3.1	63
screen-1	v-p4	48	96	24	25.0	+5.2	77
screen-2	base	48	96	21	21.9	—	72
screen-2	v-t4	48	96	18	18.8	−3.1	88
screen-2	v-t6	48	96	22	22.9	+1.0	89
screen-3	base	48	96	20	20.8	—	66
screen-3	v-a3	48	96	25	26.0	+5.2	77
screen-3	v-a4	48	96	19	19.8	−1.0	77

Table 30: Every application of the pass rule of Section 6 to a real candidate, from the frozen gate reports. Baseline and candidate are two versions of the withheld agent bundle.

	Application 1	Application 2	Application 3
Task set	small panel	large panel minus small	as application 2
Tasks; repositories; repeats	72; 18; 4	128; 19; 4	128; 19; 4
Runs per arm (all with a result row)	288	512	512
Solved runs: baseline, candidate	63, 62	103, 101	97, 118
Mean gain (points; exact fraction)	−0.35 (−1/288)	−0.39 (−1/256)	+4.10 (21/512)
(1) gain at least 2/58	not met	not met	met
Bootstrap-t; one-sided p (9,999 draws)	−0.175; 0.6034	−0.366; 0.6034	+1.954; 0.0405
One-sided p by exact enumeration	0.6113	0.6067	0.0360
(2) $p \leq 0.025$	not met	not met	not met
Repositories gaining; losing	3; 6	7; 6	9; 6
(3a) at least three repositories gain	met	met	met
Lowest leave-one-repository-out gain (points)	−1.47	−0.85	+2.94
(3b) every leave-one-out gain positive	not met	not met	met
Agent time, candidate / baseline (s)	68.4 / 66.6	89.3 / 75.6	79.4 / 72.0
(4a) ratio at most 1.2	1.026: met	1.182: met	1.103: met
Infrastructure rows: baseline, candidate	0, 0	0, 0	0, 0
(4b) infrastructure errors do not rise	met	met	met
Worst repository total (task-equivalents)	−0.50	−0.50	−0.50
(5) no repository loses two or more	met	not registered	not registered
Decision of the frozen rule	**FAIL**	**FAIL**	**FAIL**

Table 31: Standard deviation of the difference of two arms that run on the same tasks, from run-to-run variation alone. The first two rows are estimated from the repeated baseline sessions (Section 8); the third row uses the repeated public draws of unchanged 31B files [7].

Comparison of two arms	Runs per arm	SD, points	Time as run
Screen on S (48 tasks, 2 repeats)	96	2.9	1–2 hours (3–6 arms)
Confirmation on H (128 tasks, 4 repeats)	512	1.44	about 3 hours (2 arms)
Hosted, one draw per file (58 public tasks)	58	3.8	2 days

Table 32: Arms of the experiments. Labels are neutral; the category names what was changed against the arm it was built from. Prompt text and bundle files are not released.

Arm	Model	Runs	What it is
`BF5`	31B	142	Single agent: the public Budget-Fit V5 configuration, run unchanged (*reference*)
`BF5-p`	31B	30	BF5 with one changed section of the task prompt (*prompt*)
`BF5-c`	31B	32	BF5 with a prompt-only checking step before the patch is submitted (*prompt*)
`BF5-s`	31B	10	BF5 with changed sampling settings (*sampling*)
`W1`	31B	30	Wrapped: a root agent calls a coder sub-agent as a tool; the coder has its own context (*architecture*)
`W2`	31B	40	W1 with a bounded restart of the coder after a malformed tool call (*architecture*)
`W3`	31B	70	W2 with a root that can only pass the request on and restart the coder (*architecture*)
`W1-b`	31B	30	W1 with a shorter time limit, more tool calls and half the output limit per call (*budget*)
`W1-p`	31B	30	W1 with a rewritten coder prompt, a shorter time limit and half the output limit per call (*prompt*)
`SH1`	31B	30	Wrapped, coder with a shell tool only; first version (*architecture*)
`SH3`	31B	40	SH1, third version of prompt and limits (*architecture*)
`SH5`	31B	112	SH1, fifth version of prompt and limits (*architecture*)
`SH6`	31B	94	SH5 with an added analysis step in the prompt (*prompt*)
`ID24`	31B	10	Public bundle of Team Muhammed Idris, version 24, run unchanged (*reference*)
`ID24-r`	31B	10	ID24 with one file-writing tool removed, a shorter command timeout, added prompt sections (*tools*)
`e-single`	E4B	24	Single agent (*architecture*)
`e-wrap`	E4B	24	Wrapped, as W1 (*architecture*)
`e-wrap-ad`	E4B	24	e-wrap with a fine-tuned adapter on the coder (withheld) (*adapter*)
`e-BF`	E4B	64	The public single-agent prompt adapted to E4B (six tools, 28 calls) (*prompt*)
`e-BF-s`	E4B	16	e-BF with changed sampling settings (*sampling*)
`base`	E4B	1,848	The withheld E4B agent bundle; the baseline of every later comparison (*baseline*)
`base-p`	E4B	16	base with one prompt change (*prompt*)
`v-p1`	E4B	96	base with one prompt-only change (*prompt*)
`v-p2`	E4B	384	base with one prompt-only change (candidate 1) (*prompt*)
`v-p3`	E4B	96	base with one prompt-only change (*prompt*)
`v-p4`	E4B	96	base with one prompt-only change (*prompt*)
`v-b1`	E4B	96	base with a changed thinking-token limit (*budget*)
`v-t1`	E4B	15	base with a changed file-editing path, first build (*tools*)
`v-t2`	E4B	15	as v-t1, second design, first build (*tools*)
`v-t3`	E4B	15	rebuild of v-t1, option a (*tools*)
`v-t4`	E4B	111	rebuild of v-t1, option b (*tools*)
`v-t5`	E4B	15	rebuild of v-t2, option a (*tools*)
`v-t6`	E4B	623	rebuild of v-t2, option b (candidate 2) (*tools*)
`v-a1`	E4B	16	base with an added sub-agent step, first design, first build (*architecture*)
`v-a2`	E4B	16	as v-a1, second design, first build (*architecture*)
`v-a3`	E4B	608	rebuild of v-a1 (candidate 3) (*architecture*)
`v-a4`	E4B	96	rebuild of v-a2 (*architecture*)

Table 33: Every block of agent runs in the released file `runs.csv`. A block is one session on one machine; its arms ran in one job queue. Machine: one GPU with emulated L4 latency (emulated), a hosted notebook with four L4 GPUs (4×L4), or the single-GPU workstation. Source: the development tasks of the competition (dev.) or Berth. Rep.: repeats per task and arm.

Block	Model	Machine	Source	Tasks	Arms	Rep.	Runs	Note
`pub31-1`	31B	emulated	dev.	30	3	1	90	first 30 development tasks
`pub31-2`	31B	emulated	dev.	30	1	1	30	same 30 tasks
`pub31-3`	31B	emulated	dev.	30	1	1	30	same 30 tasks
`pub31-4`	31B	emulated	dev.	30	2	1	60	same 30 tasks, both arms in one queue
`pub31-5`	31B	emulated	dev.	40	2	1	80	40 further development tasks
`pub31-6`	31B	emulated	dev.	40	2	1	80	same 40 tasks, both arms in one queue
`pub31-7`	31B	emulated	dev.	70	1	1	70	all 70 tasks
`pub31-8`	31B	emulated	dev.	70	3	1	130	all 70 tasks; 30 tasks with masked test runners
`l4-1`	31B	4×L4	dev.	10	1	1	12	10 tasks, plus 2 masked
`l4-2`	31B	4×L4	dev.	24	1	1	24	24 tasks
`pubE4-1`	E4B	4×L4	dev.	24	3	1	72	24 development tasks with small fixes
`berth31-time`	31B	4×L4	Berth	10	4	1	40	10 tasks, serial
`berth31-check`	31B	4×L4	Berth	16	2	2	64	16 tasks
`berthE4-k1`	E4B	4×L4	Berth	24	2	2	96	24 tasks
`berthE4-k2`	E4B	4×L4	Berth	8	4	2	64	8 tasks
`parity-1`	E4B	workstation	Berth	24	1	1	24	no sandbox, 8 workers
`parity-2`	E4B	workstation	Berth	24	1	1	24	no sandbox, 1 worker
`parity-3`	E4B	workstation	Berth	24	1	1	24	container runner, version 1
`parity-4`	E4B	workstation	Berth	24	1	1	24	container runner, version 3
`parity-5`	E4B	workstation	Berth	24	1	1	24	container runner, version 4, run a
`parity-6`	E4B	workstation	Berth	24	1	1	24	container runner, version 4, run b
`parity-7`	E4B	workstation	Berth	24	1	1	24	container runner, version 7 (final)
`screen-1`	E4B	workstation	Berth	48	6	2	576	screen set S
`confirm-1`	E4B	workstation	Berth	72	2	4	576	small panel C
`pilot-2a`	E4B	workstation	Berth	15	2	1	30	15 tasks of the case-study panel
`pilot-2b`	E4B	workstation	Berth	15	4	1	60	same 15 tasks
`screen-2`	E4B	workstation	Berth	48	3	2	288	screen set S
`confirm-2`	E4B	workstation	Berth	128	2	4	1,024	panel H
`dev-3`	E4B	workstation	Berth	16	3	1	48	16 tasks of panel C
`screen-3`	E4B	workstation	Berth	48	3	2	288	screen set S
`confirm-3`	E4B	workstation	Berth	128	2	4	1,024	panel H, second use

Table 34: Reproduction with the released kit: the two quantities that depend on Monte Carlo draws. 95% intervals of the change against F in percentage points, and the Monte Carlo p that accompanies the exact p.

	95% interval			Monte Carlo p		
Arm	Registered	Kit, panel order	Kit command	Registered	Kit, panel order	Kit command
R	[−2.82, +2.17]	[−2.82, +2.17]	[−2.80, +2.16]	0.8650	0.8650	0.8642
C	[−2.03, +2.48]	[−2.03, +2.48]	[−2.04, +2.48]	0.9243	0.9243	0.9268
H	[−4.45, +0.38]	[−4.45, +0.38]	[−4.45, +0.25]	0.1106	0.1106	0.1095
T	[−1.42, +2.94]	[−1.42, +2.94]	[−1.42, +2.94]	0.5599	0.5599	0.5624

Table 35: The three applications of the pass rule (four repeats per task and arm). Gain: candidate minus baseline in percentage points. p: one-sided, repository-clustered. +/−: repositories with a positive or negative summed gain. LOO: lowest gain when one repository is left out. Time: ratio of mean agent time, candidate to baseline.

No.	Candidate	Tasks	Repos	Runs/arm	Base	Cand.	Gain	p	+/−	LOO	Time	Verdict
1	`v-p2`	72	18	288	63	62	−0.35	0.6034	3/6	−1.47	1.03	not confirmed
2	`v-t6`	128	19	512	103	101	−0.39	0.6034	7/6	−0.85	1.18	not confirmed
3	`v-a3`	128	19	512	97	118	+4.10	0.0405	9/6	+2.94	1.10	not confirmed

Table 36: Data files released with this paper, on the branch `paper` of the repository, in the directory `data/`.

File	Records	Content
`runs.csv`	5,024	one row per agent run of Sections 7, 8 and 10
`controls.csv`	2,494	one row per control grading (reference patch or empty patch)
`serving_bench.csv`	72	one row per cell of the serving benchmark of Section 3
`serving_servers.csv`	7	start-up time and cache size of each model server of the benchmark
`experiments.csv`	31	one row per block (Table 33)
`arms.csv`	37	one row per arm (Table 32)
`submissions.csv`	5	our hosted submissions with their displayed scores
`pass_rule_applications.json`	3	the frozen report of each application of the pass rule
`e8_runs.csv`	4,636	one row per job of the case study, with the rows of interrupted jobs
`e8_tasks.csv`	231	the tasks of the case study with their solved counts per arm
`e8_repositories.csv`	15	the same per repository
`e8_compare_table.csv`	231	the case study in the input format of the `compare` module
`e8_summary.json`	1	the estimates, intervals and p-values of the case study

Table 37: Columns of `e8_runs.csv`. The rows of interrupted jobs hold the first ten columns and the two times of day only.

Column	Content
`batch`	1 to 5: the batch of the block in which the job ran; batch 5 holds the re-runs
`stage`	1, 2 or 3: the nested stage of the job list that holds the task (118, 37 and 76 tasks)
`arm`	F, R, C, H or T
`task`	task identifier in the public task collection the panel was drawn from
`repository`	repository of the task (owner/name); the unit of clustering
`repeat`	1 to 4
`attempt`	1; 2 for the re-run of an interrupted job
`row_class`	agent: an analysed run. infrastructure: an interrupted job, not analysed
`outcome_id`	A00 solved; A01 unsolved, run completed; A02 tool-call budget reached; A05 unsolved at the five-minute limit; A06 no submission; A18 unsolved, over-long prompt rejected; I09 interrupted job
`resolved`	1 if the run is solved (the tests of the task pass on the edits the run left), else 0
`t_agent_s`	seconds of the agent phase of the job
`t_setup_s`	seconds of the setup phase, before the agent starts
`t_verify_s`	seconds of the grading phase; empty if no test was run
`model_calls`	chat requests of the run that returned a response
`model_calls_failed`	chat requests of the run that raised an error instead
`tool_calls`	tool calls made by the agent
`tokens_in`	prompt tokens, summed over the requests of the run
`tokens_out`	completion tokens, summed over the requests of the run
`max_prompt_tokens`	largest prompt of the run, in tokens
`summaries`	history summaries (compactions) made during the run
`patch_chars`	characters of the diff the run left; 0 if none
`test_exit`	exit status of the test command; 0 = tests passed; −1 = no test was run
`start_utc`	start of the job (UTC)
`end_utc`	end of the job (UTC)
`requests_checked`	returned requests of the run that the frozen checker re-rendered
`requests_changed`	of them, requests whose prompt the reference rule of arm F would render differently (the exposure count; 0 in F)
`check_passed`	1 if the run passed the prompt check